

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN
SYSTEM AUTOMATION

Môn học: Quản trị mạng và hệ thống

Mã lớp: NT132.Q11.ANTT

Giảng viên hướng dẫn: ThS. Trần Thị Dung

Thực hiện bởi nhóm 15, bao gồm:

1. Vương Thành Đạt – 23520281
2. Trần Khánh Toàn – 23521610
3. Nguyễn Bá Quân – 23521260

TP. Hồ Chí Minh, tháng 12 năm 2025



MỤC LỤC

DANH MỤC HÌNH ẢNH.....	3
DANH MỤC BẢNG.....	4
1. TỔNG QUAN.....	5
1.1. Giới thiệu.....	5
1.2. Thành phần.....	5
1.3. Hoạt động.....	7
2. TRIỂN KHAI.....	9
2.1. Mô hình.....	9
2.2. Cài đặt.....	10
2.2.1. Cài đặt Ansible trên Control Node.....	10
2.2.2. Cấu hình file config.....	11
2.2.3. Cấu hình file hosts.....	12
2.2.4. Cài đặt trên Managed Node.....	13
2.3. Cấu hình.....	14
2.3.1. Cài đặt biến trong Ansible.....	14
2.3.1.1. all.yml.....	14
2.3.1.2. backup.yml.....	15
2.3.1.3. db.yml.....	15
2.3.1.4. haproxy.yml.....	16
2.3.1.5. redis.yml.....	16
2.3.1.6. web.yml.....	16
2.3.2. Common.....	17
2.3.2.1. Tasks/main.yml.....	17
2.3.2.2. Tasks/security.yml.....	18
2.3.2.3. Handler.....	21
2.3.3. Database.....	21
2.3.4. Haproxy.....	24
2.3.4.1. Tasks/main.yml.....	24
2.3.4.2. Handlers/main.yml.....	25
2.3.4.3. Templates/haproxy.cfg.j2.....	26
2.3.5. Redis.....	28
2.3.5.1. Redis/Tasks/main.yml.....	28
2.3.6. Web.....	29
2.3.6.1. tasks/main.yml.....	29
2.3.6.2. xtemplates/vhost.conf.j2.....	31
2.3.7. Wordpress.....	32
2.3.7.1. tasks/main.yml.....	32
2.3.7.2. templates/wp-config.php.j2.....	34
2.3.8. Backup.....	36
2.3.8.1. tasks/main.yml.....	36



2.3.9. Playbooks.....	40
3. KẾT QUẢ VÀ KẾT LUẬN	41
3.1. Kết quả	41
3.2. Kết luận.....	53
4. PHỤ LỤC.....	54
4.1. Tự đánh giá.....	54
4.2. Phân công.....	54
4.3. Trả lời câu hỏi	54
TÀI LIỆU THAM KHẢO	57



DANH MỤC HÌNH ẢNH

Hình 1.2.1. Kiến trúc Ansible (tổng thể).....	5
Hình 1.2.2. Kiến trúc Ansible (chi tiết).....	6
Hình 1.3.1. Sơ đồ hoạt động của ansible	7
Hình 2.1.1. Mô hình docker ban đầu.....	9
Hình 2.1.2. Mô hình VMWare lúc sau.....	9
Hình 2.1.3. Luồng hoạt động	10
Hình 2.2.1. Kết quả ping host	13
Hình 3.1.1. Kết quả chạy lệnh docker compose up.....	41
Hình 3.1.2. Chi tiết cấu hình php.....	41
Hình 3.1.3. Kết quả khởi động playbook.....	42
Hình 3.1.4. Kết quả khởi động database.....	42
Hình 3.1.5. Trang WordPress sau khi build	43
Hình 3.1.6. Đăng kí tài khoản (1).....	43
Hình 3.1.7. Đăng kí tài khoản (2).....	44
Hình 3.1.8. Dashboard của WordPress	44
Hình 3.1.9. Đăng nhập	45
Hình 3.1.10. Giao diện trang web.....	45
Hình 3.1.11. Trạng thái của HAProxy	45
Hình 3.1.12. Tải và cài đặt gói cần thiết.....	46
Hình 3.1.13. Setup controller	46
Hình 3.1.14. Các bước cấu hình ssh key	46
Hình 3.1.15. Cấu hình mật khẩu cho ansible và copy ssh-id cho kết nối.....	47
Hình 3.1.16. Kiểm tra kết nối.....	47
Hình 3.1.17. Kết quả sau khi chạy.....	48
Hình 3.1.18. Localhost test browser.....	48
Hình 3.1.19. Kiểm tra bằng ip.....	49
Hình 3.1.20. Kiểm tra cơ sở dữ liệu	49
Hình 3.1.21. Kiểm tra redis hoạt động.....	50
Hình 3.1.22. Kiểm tra backup.....	50
Hình 3.1.23. Đăng nhập user vào haproxy	50
Hình 3.1.24. Trạng thái hoạt động của 2 web server	51
Hình 3.1.25. Đăng kí	51
Hình 3.1.26. Đăng nhập	52
Hình 3.1.27. Trạng thái đã hoạt động trên 2 web server	52



DANH MỤC BẢNG

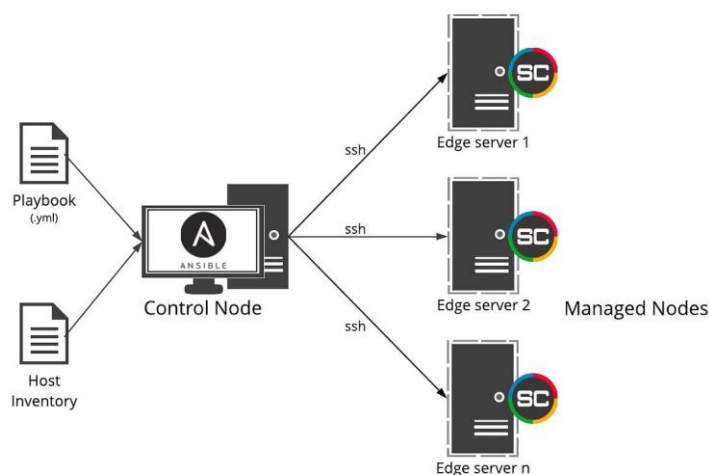
Bảng 4.1.1. Tự đánh giá.....	54
Bảng 4.2.1. Phân công	54
Bảng 4.3.1. Trả lời câu hỏi.....	56

1. TỔNG QUAN

1.1. Giới thiệu

- Ansible là một ứng dụng mã nguồn mở giúp tự động hóa các công việc lặp đi lặp lại ở trên các hệ thống từ xa như: quản lý cấu hình, triển khai ứng dụng, cung cấp tài nguyên và điều phối các luồng công việc phức tạp.
- Ansible được thiết kế để có thể cấu hình trên các hệ thống Unix và Window. Ansible không cần agent mà sử dụng các kết nối từ xa như SSH hoặc Window Remote Management. Nút điều khiển của Ansible chạy trên hầu hết các hệ thống Unix có thể chạy Python, bao gồm cả Windows nếu đã cài đặt Windows Subsystem for Linux. Cấu hình hệ thống được định nghĩa một phần bằng cách sử dụng ngôn ngữ YAML, đây là một ngôn ngữ khai báo dữ liệu và được thiết kế để con người dễ dàng đọc.

1.2. Thành phần

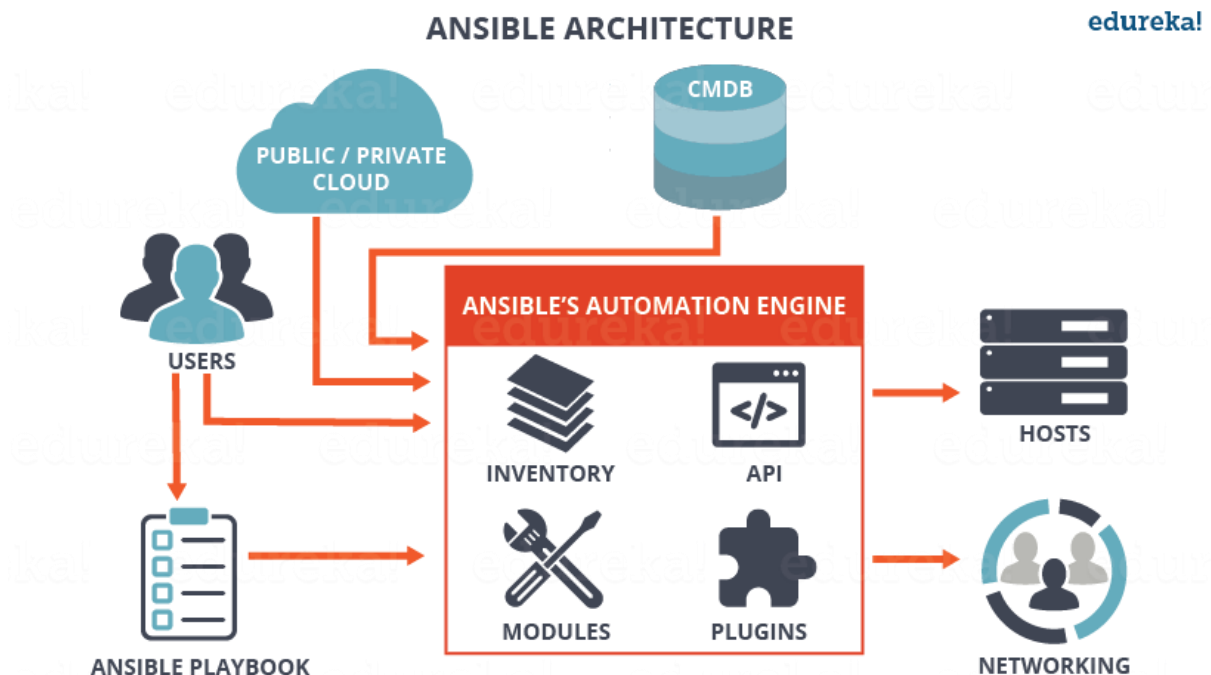


Hình 1.2.1. Kiến trúc Ansible (tổng thể)¹

- Kiến trúc của Ansible được xây dựng dựa trên mô hình client-server. Tại trung tâm của hệ thống là Control Node, đây là máy chủ nơi Ansible được cài đặt và từ đó người quản trị thực thi các lệnh automation. Control Node phải chạy trên hệ điều hành Linux hoặc Unix, và nó kết nối đến các Managed Nodes (các máy chủ được quản lý) thông qua giao thức SSH đối với Linux hoặc WinRM đối với Windows. Điểm mạnh của kiến trúc này là các Managed Nodes không cần cài đặt bất kỳ phần mềm đặc biệt nào, chỉ cần có SSH và Python - những thành phần thường có sẵn trên hầu hết các hệ thống Linux hiện đại. Các thành phần của Ansible bao gồm:

- **Control Node:** Control Node đóng vai trò như một trung tâm điều phối lệnh, nơi toàn bộ các luồng công việc tự động được khởi tạo và điều phối. Đây là máy chủ hoặc workstation nơi Ansible được cài đặt và từ đó người quản trị thực thi các playbooks, các lệnh, và quản lý toàn bộ hạ tầng.
- **Managed Node:** Đây là các máy chủ, thiết bị, hoặc máy ảo mục tiêu mà Ansible cấu hình và quản lý. Ansible kết nối với các nút này, thường thông qua giao thức SSH đối với các hệ thống Linux/Unix hoặc WinRM đối với Windows, để thực thi các tác vụ. Tập tin Inventory trên nút điều khiển (Control node) định nghĩa và nhóm các nút được quản lý này.

¹Kiến trúc Ansible (tổng thể)



Hình 1.2.2. Kiến trúc Ansible (chi tiết)²

○ **Inventory:** Inventory đóng vai trò là nơi định nghĩa tất cả các máy chủ và thiết bị mà Ansible quản lý. Inventory có thể được viết dưới dạng file tĩnh với định dạng INI hoặc YAML, cho phép chia các hosts thành các nhóm dựa trên chức năng, môi trường. Bên cạnh inventory tĩnh, Ansible còn hỗ trợ inventory động, sử dụng các scripts hoặc plugins để tự động truy vấn và lấy thông tin hosts từ các nguồn bên ngoài như AWS, Azure, Google Cloud Platform,... Khi chạy playbook, Ansible thực thi inventory script để lấy danh sách host mới nhất, đảm bảo luôn hoạt động với thông tin chính xác. Ngoài việc định nghĩa hosts, inventory còn cho phép gán biến cho từng host hoặc nhóm. Những biến này có thể chứa thông tin như port kết nối SSH, credentials, hoặc các tham số cấu hình đặc thù cho từng môi trường.

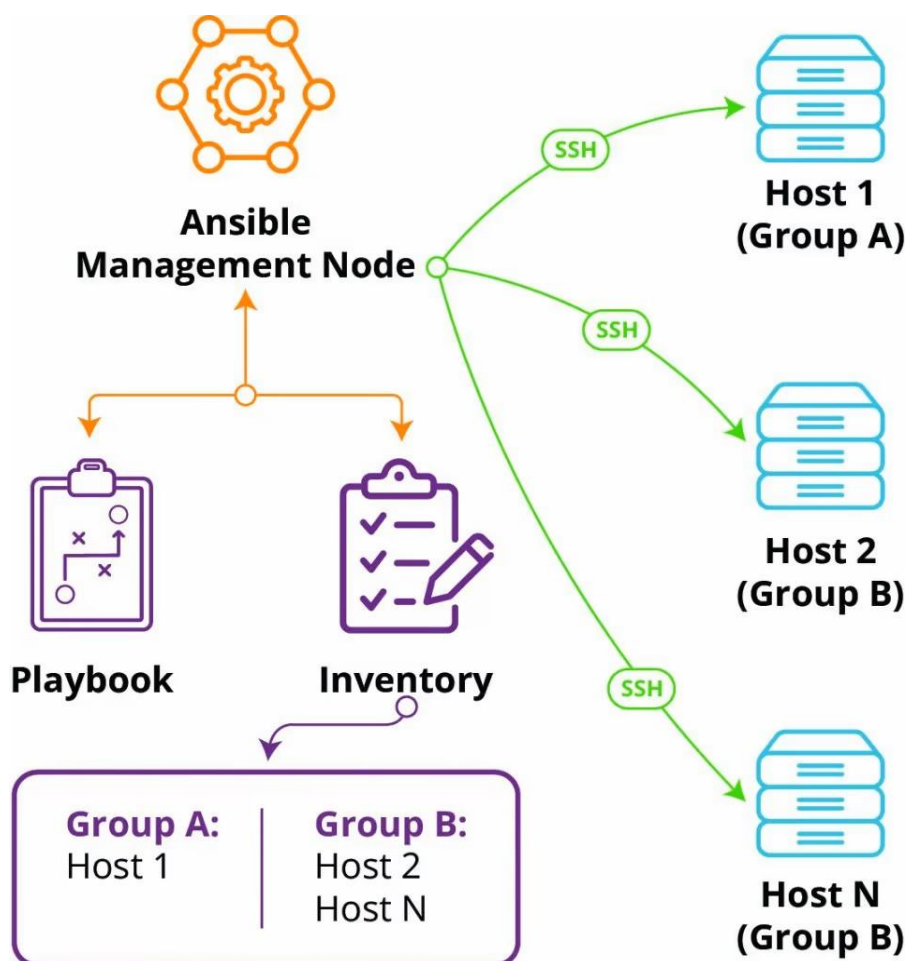
○ **Playbook:** Playbooks là một thành phần quan trọng của Ansible, nơi định nghĩa toàn bộ luồng công việc tự động dưới dạng code. Playbooks được viết bằng YAML, một định dạng dễ đọc và dễ viết, cho phép mô tả các bước automation một cách rõ ràng. Mỗi playbook chứa một hoặc nhiều plays, và mỗi play định nghĩa một tập hợp các tasks được thực thi trên một nhóm hosts cụ thể. Playbooks không chỉ đơn thuần là danh sách các lệnh tuần tự, mà còn hỗ trợ điều kiện logic, vòng lặp, xử lý lỗi, và khả năng tổ chức code thành các thành phần tái sử dụng được. Cách tiếp cận này cho phép người dùng mô tả trạng thái mong muốn của hệ thống thay vì liệt kê các bước thực hiện, giúp code tự động trở nên dễ hiểu và dễ bảo trì hơn. Ngoài ra để Playbook dễ quản lý hơn, còn có thêm:

- **Roles:** Roles là cơ chế tổ chức code trong Ansible theo cấu trúc có hệ thống, giúp chia nhỏ các playbooks phức tạp thành các thành phần logic, dễ quản lý và tái sử dụng. Mỗi role có cấu trúc thư mục với các thư mục con như tasks, handlers, templates, files,... Cách tổ chức này không chỉ giúp code trở nên rõ ràng hơn mà còn cho phép chia sẻ roles giữa các dự án và nhóm.

²Kiến trúc Ansible (chi tiết)

- **Tasks:** Task đại diện cho một hành động cụ thể cần thực hiện. Mỗi task kết hợp một module với các tham số cụ thể và metadata mô tả task đó. Một task điển hình bao gồm nhiều thành phần. Name là mô tả dễ hiểu về task đó, module name định nghĩa hành động cụ thể, module nào được gọi và với tham số gì,...
- **Modules:** Modules là những đơn vị công việc nhỏ nhất, mỗi module được thiết kế để thực hiện một việc cụ thể. Ansible cung cấp hàng nghìn modules có sẵn cho các tác vụ phổ biến (quản lý package, system,...).
- **API:** API là để giúp tương tác với thành phần cốt lõi của Ansible, có thể dùng nó để viết các plugins hoặc modules. Ngoài ra, API còn cho phép các hệ thống bên ngoài ra lệnh cho Ansible.
- **Plugin:** Plugin đóng vai trò như là một tiện ích mở rộng, hỗ trợ thêm nhiều tính năng khác cho Ansible. Các Plugin có chức năng tiêu biểu như tự lấy thông tin hosts từ bên ngoài, xác định cách kết nối đến Managed Host,...
- **CMDB:** Configuration Management Database, là một cơ sở dữ liệu lưu trữ các thông tin chi tiết về cơ sở hạ tầng của các tổ chức, giúp đảm bảo Ansible luôn hoạt động trên danh sách các Managed Host chính xác và mới nhất.

1.3. Hoạt động



Hình 1.3.1. Sơ đồ hoạt động của ansible³

³Sơ đồ hoạt động của ansible

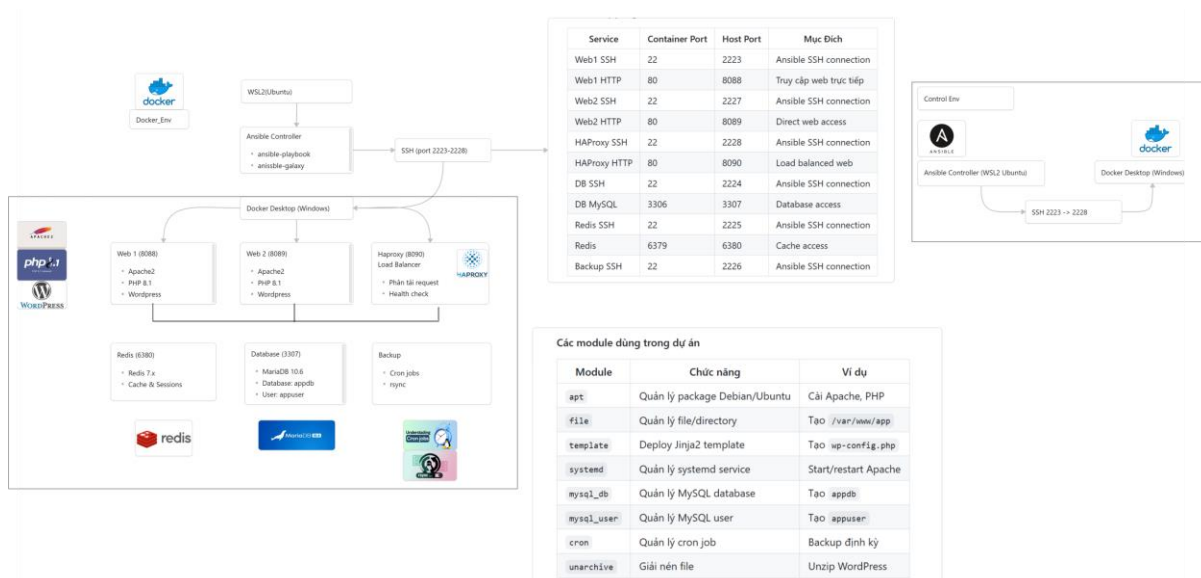


- Ansible hoạt động theo một quy trình đơn giản, bắt đầu từ Control Node, máy chủ nơi Ansible được cài đặt và người quản trị thực thi các lệnh. Khi một playbook được chạy, Ansible đầu tiên đọc file inventory để xác định danh sách các Managed Nodes cần quản lý. Inventory có thể là file tĩnh hoặc được lấy động từ cloud providers, giúp Ansible biết chính xác cần kết nối đến đâu và với cấu hình gì.
- Sau khi xác định hosts mục tiêu, Ansible thiết lập kết nối SSH đến từng Managed Node. Đối với mỗi task trong playbook, Ansible sinh ra một đoạn mã Python trên Control Node, sau đó copy đoạn mã này lên Managed Node thông qua SSH connection. Module được thực thi cục bộ trên Managed Node, thực hiện công việc cụ thể như cài đặt package, sửa file cấu hình, hoặc khởi động service.
- Sau khi hoàn thành, module trả về kết quả dưới dạng JSON chứa thông tin về trạng thái thực thi về Control Node. Ansible thu thập kết quả này, xóa các file tạm trên Managed Node, và đóng SSH connection. Toàn bộ quá trình này được lặp lại song song trên nhiều host, với số lượng kết nối đồng thời được kiểm soát thông qua tham số forks để tối ưu hiệu suất.
- Nếu có tasks thông báo handlers, các handlers này không chạy ngay mà được xếp hàng đợi lại và chỉ thực thi một lần duy nhất vào cuối play, sau khi tất cả tasks hoàn thành. Cơ chế này tránh việc restart service nhiều lần không cần thiết. Ansible cũng tự động thu thập facts - thông tin về hệ thống như OS version, IP address, RAM - từ Managed Nodes, cho phép playbooks đưa ra quyết định thông minh dựa trên đặc điểm của từng hệ thống. Toàn bộ luồng công việc này diễn ra nhanh chóng, hiệu quả, và không để lại bất kỳ process hoặc agent nào chạy trên Managed Nodes sau khi hoàn tất.

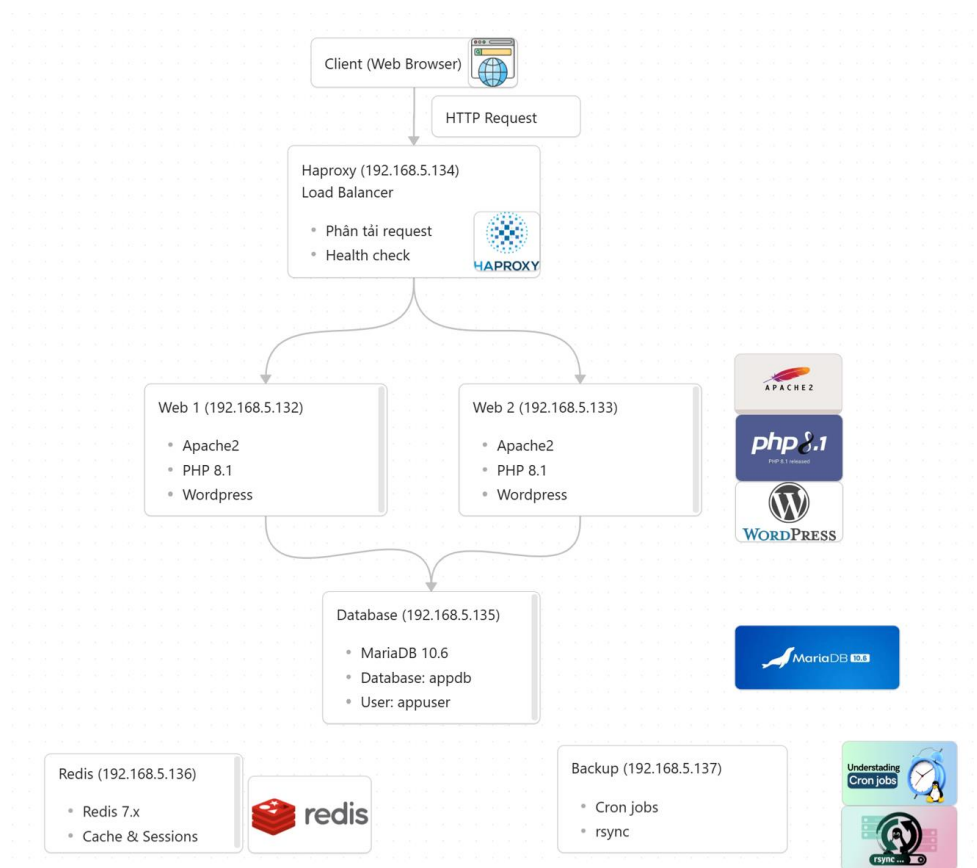


2. TRIỂN KHAI

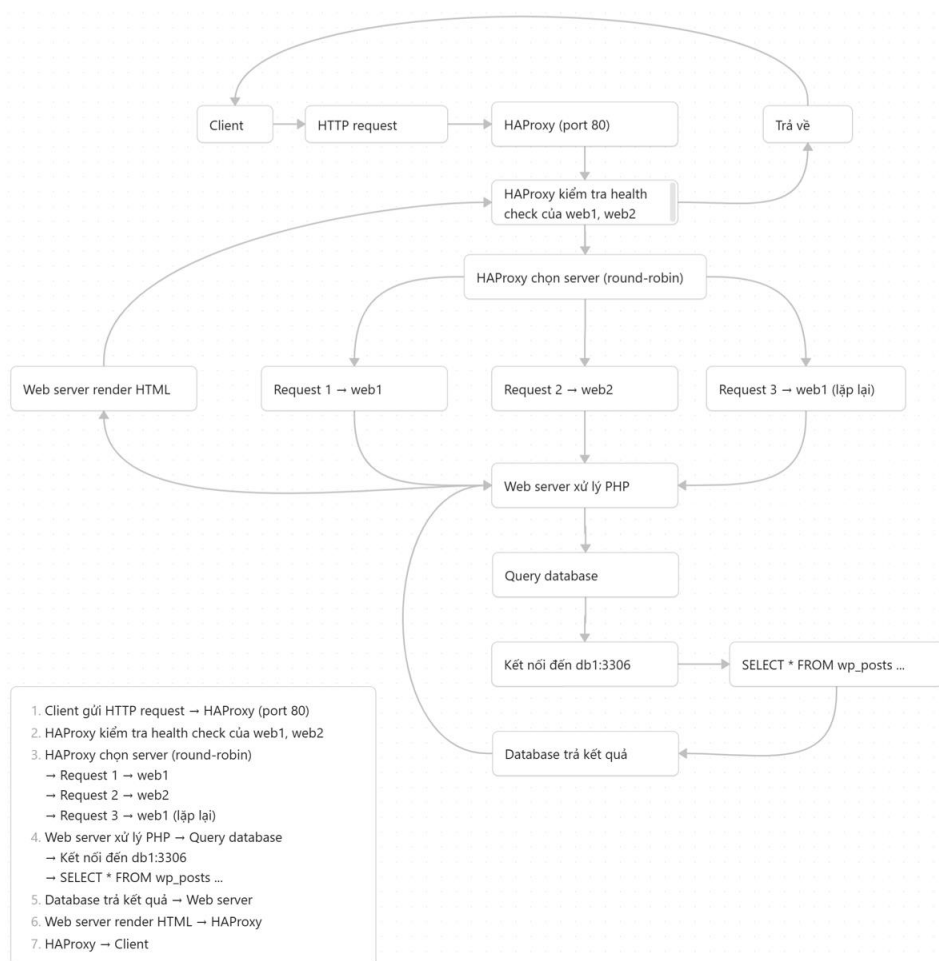
2.1. Mô hình



Hình 2.1.1. Mô hình docker ban đầu



Hình 2.1.2. Mô hình VMWare lúc sau



Hình 2.1.3. Luồng hoạt động

2.2. Cài đặt

2.2.1. Cài đặt Ansible trên Control Node

- Cập nhật kho cài đặt và cài ansible:

```
sudo apt update && sudo apt install -y ansible
```

- Cài đặt docker với ansible-galaxy:

```
ansible-galaxy collection install community.docker
```

- Cài đặt môi trường ảo python:

```
sudo apt update -y && sudo apt install -y python3-venv python3-pip sshpass  
python3 -m venv "$HOME/.venvs/ansible"  
source "$HOME/.venvs/ansible/bin/activate"  
pip install --upgrade pip && pip install ansible==8.7.0
```



- Cài đặt các modules cho Ansible:

```
ansible-galaxy collection install -r  
collections/requirements.yml
```

2.2.2. Cấu hình file config

```
[defaults]  
inventory = inventories/docker/hosts.ini  
host_key_checking = False  
retry_files_enabled = False  
forks = 10  
timeout = 30  
interpreter_python = auto_silent  
stdout_callback = yaml  
bin_ansible_callbacks = True  
  
[ssh_connection]  
pipelining = True  
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

File cấu hình này chia các biến ra thành 2 nhóm:

- **defaults:**

- **inventory:** Chứa đường dẫn đến file inventory của Ansible, ở đây là file hosts.ini.
- **host_key_checking:** Kiểm tra khóa máy chủ SSH, đặt là False để không phải kiểm tra tránh làm gián đoạn quá trình tự động hóa.
- **retry_files_enabled:** Tắt tính tạo tệp retry khi playbook thất bại, tệp retry là để chứa danh sách host bị lỗi.
- **forks:** Đặt số lượng tiến trình, ở đây Ansible có thể quản lý 10 host cùng lúc.
- **timeout:** Thời gian chờ tối đa là 30 giây cho các kết nối SSH.
- **stdout_callback:** Chỉ định dạng đầu ra trên terminal là yaml.
- **bin_ansible_callbacks:** Cho phép sử dụng các plugin callback trong quá trình chạy lệnh.

- **ssh_connection:**

- **pipelining:** Kích hoạt pipelining để tăng tốc độ chạy, thay vì mở nhiều kết nối SSH cho mỗi lệnh, Ansible cố gắng gửi nhiều lệnh qua một kết nối SSH duy nhất.
- **ssh_args:** Thiết lập cơ chế SSH Multiplexing, cho phép kết nối SSH đầu tiên trở thành một kênh điều khiển và giữ kênh đó mở thêm 60 giây sau khi Ansible hoàn thành, giúp các Task tiếp theo trong Playbook có thể tái sử dụng cùng một kết nối SSH.



2.2.3. Cấu hình file hosts

- Cấu hình file host gồm địa chỉ các host, username, file key's path và biến toàn cục:

```
[web]
web1      ansible_host=192.168.5.132      ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/id_rsa
web2      ansible_host=192.168.5.133      ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/id_rsa

[haproxy]
haproxy1  ansible_host=192.168.5.134      ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/id_rsa

[db]
db1       ansible_host=192.168.5.135      ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/id_rsa

[redis]
redis1    ansible_host=192.168.5.136      ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/id_rsa

[backup]
backup1   ansible_host=192.168.5.137      ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/id_rsa

[all:vars]
ansible_connection=ssh
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null'
ansible_remote_tmp=/tmp
```

- Chia các máy chủ ra thành từng nhóm để dễ dàng sử dụng cho mục đích khác nhau như [web], [db].
- Đầu tiên là cấu hình web, gán địa chỉ của máy mà mình muốn cài đặt trang web vào biến `ansible_host`, tên user vào biến `ansible_user` và đường dẫn đến file khóa tại `ansible_ssh_private_key_file`.
- Tương tự với các nhóm máy chủ khác, cũng thêm biến cho IP host, username và đường dẫn đến file khóa.



- Nhóm cuối cùng là để xác định các biến toàn cục:
 - o **ansible_connection**: Khai báo phương thức kết nối sử dụng là SSH
 - o **ansible_python_interpreter**: Khai báo đường dẫn đến trình biên dịch Python trên Managed Node, Ansible yêu cầu python để chạy Module.
 - o **ansible_ssh_common_args**: Thêm các biến tại Control Node, ở đây thêm 2 biến là StrictHostKeyChecking= no (Vô hiệu hóa kiểm tra khóa của Managed Node) và UserKnownHostsFile=/dev/null (Tránh lưu trữ khóa máy chủ trong known_hosts).
 - o **ansible_remote_tmp**: Đường dẫn đến thư mục tạm thời, nơi Ansible thực thi playbooks, khi hoàn thành dọn dẹp thư mục này.
- Thử ping các host sau khi đã cấu hình:

```
$ ansible -i inventories/staging/hosts.ini all -m ping -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [Ansible Ad-Hoc] *****

TASK [ping] *****
ok: [backup1]
ok: [web1]
ok: [db1]
ok: [redis1]

PLAY RECAP *****
backup1 : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
db1      : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
redis1   : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
web1     : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Hình 2.2.1. Kết quả ping host

2.2.4. Cài đặt trên Managed Node

- Kiểm tra trên Managed Node dịch vụ SSH có đang bật và sẵn sàng để nhận kết nối SSH:

```
sudo systemctl status sshd
```

- Nếu dịch vụ chưa được bật thì bật bằng câu lệnh:

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

- Ngoài ra kiểm tra xem tường lửa chặn cổng 22 không:

```
sudo ufw status
sudo ufw allow ssh
```



2.3. Cấu hình

2.3.1. Cài đặt biến trong Ansible

- Các file này định nghĩa biến này đều được sử dụng trong các file task ở dưới, còn ở đây chỉ khai báo các biến trước. Các biến được sử dụng dưới dạng `{{ var }}`.

2.3.1.1. *all.yml*

```
apt_package_state: present

web_http_port: 80
db_port: 3306
redis_port: 6379

web_app_root: /var/www/app
web_server_name: localhost

db_name: appdb
db_user: appuser
db_password: "appsecret!"

allow_world_readable_tmpfiles: true

wordpress_db_host:                                     "{{
hostvars[groups['db'][0]]['ansible_host'] }}" :3306"

haproxy_backend_web1:    "{{ hostvars['web1']['ansible_host']
}}:80"
haproxy_backend_web2:    "{{ hostvars['web2']['ansible_host']
}}:80"
```

- **apt_package_state**: Chỉ định trạng thái của các gói phần mềm, present nghĩa là các gói phải được cài đặt.
- **web_http_port, db_port, redis_port**: Định nghĩa port cho web, database và redis theo các port mặc định cho từng dịch vụ. Web lắng nghe kết nối HTTP qua cổng 80, 3306 là cổng mặc định cho MariaDB và 6379 là cổng mặc định của Redis.
- **web_app_root**: Đường dẫn thư mục gốc, nơi mã nguồn web được triển khai và phục vụ.
- **web_server_name**: Tên máy chủ được đặt trong cấu hình máy chủ web.
- **db_name**: Tên của cơ sở dữ liệu.
- **db_user**: Tên người dùng mà web sử dụng để kết nối và thao tác với cơ sở dữ liệu.
- **db_password**: Mật khẩu cho người dùng.
- **allow_world_readable_tmpfiles**: Biến này thường được dùng để cho ứng dụng truy cập các tệp tạm thời trong /tmp, tùy thuộc vào module hoặc ứng dụng.



- **wordpress_db_host**: Tự động lấy IP của máy chủ cơ sở dữ liệu trong file hosts.ini và nối với cổng 3306. Biến này được sử dụng để web biết nó phải kết nối đến DB nào.
- **haproxy_backend_web1, haproxy_backend_web2**: Tự động lấy IP của máy chủ web1/web2 trong file hosts.ini và nối với cổng 80. Biến này định nghĩa một máy chủ backend cho cấu hình HAProxy.

2.3.1.2. backup.yml

```
backup_cron_enabled: true
backup_dir: /backup
backup_time: "0 3 * * *" # 3h sáng
backup_test_enabled: false

backup_web_hosts: []
backup_db_host: "{{ hostvars[groups['db'][0]]['ansible_host']
}}:3306"
backup_db_user: "root"
backup_db_password: ""
```

- **backup_cron_enabled**: Dùng để Bật/Tắt tác vụ Cron sao lưu. Được đặt là true để Ansible tạo hoặc kích hoạt một công việc định kỳ để chạy tác vụ sao lưu.
- **backup_dir**: Biến này để chỉ định đường dẫn thư mục trên máy chủ sao lưu, nơi tất cả các tệp sao lưu được lưu trữ.
- **backup_time**: Để lên một chu trình cho Cron, luôn sao lưu vào 3h sáng mỗi ngày (Minute/Hour/Day/Month/DayInWeek)
- **backup_test_enabled**: Để Bật/Tắt chế độ kiểm tra. Nếu được đặt là true, tác vụ sao lưu có thể chỉ chạy ở chế độ thử nghiệm (không tạo bản sao lưu hoàn chỉnh).
- **backup_web_hosts**: Danh sách các máy chủ web cần sao lưu.
- **backup_db_host**: Địa chỉ và cổng của máy chủ DB cần sao lưu. Tương tự như trước, nó tra cứu IP của host đầu tiên trong nhóm [db] và nối với cổng MariaDB (3306) để xác định nguồn dữ liệu DB.
- **backup_db_user**: Tên người dùng DB được sử dụng để kết nối và xuất dữ liệu từ cơ sở dữ liệu.
- **backup_db_password**: Mật khẩu cho người dùng DB.

2.3.1.3. db.yml

```
mariadb_bind_address: 0.0.0.0
mariadb_port: "3306"
```

- **mariadb_bind_address**: Giá trị này để xác định địa chỉ mà MariaDB lắng nghe. Ở đây để "0.0.0.0" nghĩa là MariaDB lắng nghe tất cả các địa chỉ.
- **mariadb_port**: Định nghĩa port mà Maria đang chạy.

2.3.1.4. *haproxy.yml*

```
haproxy_stats_password: "admin123"  
haproxy_frontend_port: "80"
```

- **haproxy_stats_password**: Là mật khẩu để truy cập vào trang thống kê của HAProxy.
- **haproxy_frontend_port**: Đây là cổng mà HAProxy nhận kết nối của người dùng.

2.3.1.5. *redis.yml*

```
redis_bind_address: 0.0.0.0  
redis_tcp_port: "6379"
```

- **redis_bind_address**: Địa chỉ mà Redis lắng nghe. Đặt giá trị là "0.0.0.0" có nghĩa là Redis lắng nghe tất cả các địa chỉ.
- **redis_tcp_port**: Đây là cổng TCP mà dịch vụ Redis đang chạy.

2.3.1.6. *web.yml*

```
apache_listen_port: "80"  
php_packages:  
- php  
- libapache2-mod-php  
- php-mysql  
- php-cli  
- php-curl  
- php-xml  
- php-gd  
- php-zip  
  
wordpress_install: true  
wordpress_set_owner: true
```

- **apache_listen_port**: Là cổng lắng nghe của Apache.
- **php_packages**: Là danh sách các gói PHP cần thiết. Ansible sử dụng module quản lý gói (như apt hoặc yum) để đảm bảo các gói này được cài đặt trên các máy chủ web.
- **wordpress_install**: Để Bật/Tắt cài đặt WordPress. Đặt là true để Ansible tải xuống, giải nén và cấu hình các tệp của WordPress trên máy chủ.
- **wordpress_set_owner**: Để Thiết lập quyền sở hữu tệp. Nếu là true, Ansible đảm bảo các tệp và thư mục của WordPress được đặt quyền sở hữu chính xác cho người dùng dịch vụ web.

2.3.2. Common

2.3.2.1. Tasks/main.yml

```
- name: Update apt cache
  apt:
    update_cache: true
```

- Task Update apt cache:
 - o **name:** Để đặt tên cho Task.
 - o **apt:** Đây là việc sử dụng Module apt để tải các tập tin cần thiết.
 - o **update_cache:** Tham số này là để module apt sử dụng lệnh tương đương với “sudo apt update”, cập nhật danh sách gói phần mềm mới nhất.

```
- name: Install base packages
  apt:
    name:
      - curl
      - git
      - unzip
      - ca-certificates
    state: "{{ apt_package_state }}"
```

- Task Install base packages:
 - o **name:** Liệt kê các gói phần mềm cần được quản lý.
 - o **state:** Thiết lập trạng thái của gói tin, như đã được khai báo biến ở file all.yml thì giá trị này là present, nghĩa là gói tin phải được cài đặt.

```
- name: Include security hardening tasks
  include_tasks: security.yml
  when: security_enabled | default(false) | bool
```

- Task Include security hardening task:
 - o **include_tasks:** Yêu cầu Ansible thực thi tất cả các Task trong file security.yml.
 - o **when:** Để kiểm tra điều kiện có nên thực thi task này hay không. Nếu muốn cài đặt thì có thể đặt biến “security_enabled: true” ở trong file, còn lại mặc định là không cài.

2.3.2.2. *Tasks/security.yml*

```
- name: Update system packages for security
  apt:
    update_cache: true
    upgrade: dist
    autoremove: true
    autoclean: true
  when: security_auto_update | default(false) | bool
```

- Task Update system packages for security:

- **update_cache**: Tương đương “sudo apt update”.
- **upgrade**: Tham số là dist, tương đương với lệnh “sudo apt dist-upgrade”, để nâng cấp hệ thống một cách toàn diện.
- **autoremove**: Tương đương với lệnh “sudo apt autoremove”, để tự động xóa các phần mềm không có nhu cầu dùng.
- **autoclean**: Tương đương với lệnh “sudo apt autoclean”, xóa các tệp gói (.deb) không còn được hỗ trợ và không còn khả dụng.
- **when**: Điều kiện chạy là khi biến security_auto_update được đặt là true, còn lại mặc định là không chạy.

```
- name: Configure UFW firewall (if enabled)
  ufw:
    rule: "{{ item.rule }}"
    port: "{{ item.port }}"
    proto: "{{ item.proto | default('tcp') }}"
    state: "{{ item.state | default('enabled') }}"
  loop: "{{ security_firewall_rules | default([]) }}"
  when: security_firewall_enabled | default(false) | bool
  ignore_errors: true
```

- Task Configure UFW firewall (if enabled):

- **ufw**: Đây là Module để quản lý tường lửa trên Debian/Ubuntu.
- **rule**: Đặt các loại quy tắc cho tường lửa, các quy tắc này người dùng có thể tự cài bằng cách thêm biến cho Ansible.
- **port**: Đây là cổng nơi mà các quy tắc ở trên áp dụng.
- **proto**: Đặt giao thức, nếu không được định nghĩa, mặc định là tcp.
- **state**: Đây là trạng thái của tường lửa, nếu không được định nghĩa, mặc định là enabled.
- **loop**: Task lặp lại mọi phần tử trong danh sách được định nghĩa bởi biến security_firewall_rules. Nếu biến này không được định nghĩa, nó mặc định là một danh sách rỗng, khiến Task không chạy.
- **when**: Task chỉ chạy khi biến security_firewall_enabled được đặt là true.
- **ignore_errors**: Nếu Task gặp lỗi, Ansible bỏ qua lỗi và tiếp tục chạy.

```
- name: Disable root SSH login
lineinfile:
  path: /etc/ssh/sshd_config
  regexp: "^PermitRootLogin"
  line: "PermitRootLogin no"
  state: present
notify: restart sshd
when: security_disable_root_ssh | default(true) | bool
```

- Task Disable root SSH login:

- **lineinfile**: Module này đảm bảo một dòng cụ thể trong một tệp văn bản có tồn tại, thay đổi hoặc bị xóa không.
- **path**: Tham số này để chỉ định đường dẫn tới file cần sửa đổi.
- **regexp**: Tham số này có biến là một Regex, nó tìm kiếm dòng bắt đầu có chữ PermitRootLogin.
- **line**: Tham số này để thay đổi hoặc chèn sao cho khớp với Regex.
- **state**: Tham số đảm bảo dòng cấu hình phải có mặt trong tệp.
- **notify**: Tham số này là notify, để thực hiện một hành động Handler, ở đây Handler được thực thi là restart sshd.
- **when**: Điều kiện kích hoạt mặc định là luôn kích hoạt, nếu không muốn có thể đặt biến Ansible “security_disable_root_ssh” là false.

- Tóm lại nhiệm vụ của Task này là để vô hiệu hóa việc đăng nhập trực tiếp bằng tài khoản root qua SSH trên Managed Node.

```
- name: Disable password authentication (use keys only)
lineinfile:
  path: /etc/ssh/sshd_config
  regexp: "^PasswordAuthentication"
  line: "PasswordAuthentication no"
  state: present
notify: restart sshd
when: security_ssh_key_only | default(false) | bool
```

- Task Disable password authentication (use keys only):

- Tương tự như Task ở trên, Task này mục đích để vô hiệu hóa việc sử dụng mật khẩu để xác thực SSH trên Managed Node.



```
- name: Set SSH key for ansible user
authorized_key:
  user: ansible
  state: present
  key: "{{ item }}"
loop: "{{ security_ssh_public_keys | default([]) }}"
when: security_ssh_keys | default([]) | length > 0
```

- Task Set SSH key for ansible user:

- **authorized_key**: Module này dùng để quản lý khóa SSH công khai.
- **user**: Tham số xác định người dùng mà khóa SSH được thêm vào.
- **state**: Tham số để đảm bảo sự có mặt của khóa công khai SSH.
- **key**: Tham số để lấy khóa công khai trong các phần tử của security_ssh_keys.
- **loop**: Vòng lặp để lấy danh sách khóa công khai.
- **when**: Chỉ chạy task khi biến security_ssh_keys có nhiều hơn 0 phần tử, còn lại mặc định là danh sách rỗng.

```
- name: Remove unnecessary packages
apt:
  name: "{{ item }}"
  state: absent
  purge: true
loop: "{{ security_remove_packages | default([]) }}"
when: security_remove_packages | default([]) | length > 0
```

- Task Remove unnecessary packages:

- **name**: Tham số này là tên của các phần mềm cần thao tác. Được lấy trong các phần tử của security_remove_packages.
- **state**: Tham số được đặt là absent, đảm bảo gói phần mềm không tồn tại.
- **purge**: Tham số này yêu cầu không chỉ xóa gói mà còn xóa cả tệp cấu hình, xóa một cách triệt để.
- **loop**: Vòng lặp lấy danh sách gói cần xóa
- **when**: Chỉ chạy task khi biến security_remove_packages có nhiều hơn 0 phần tử, còn lại mặc định là danh sách rỗng.

```
- name: Set restrictive file permissions for sensitive files
file:
  path: "{{ item.path }}"
  mode: "{{ item.mode | default('0600') }}"
  owner: "{{ item.owner | default('root') }}"
  group: "{{ item.group | default('root') }}"
loop: "{{ security_sensitive_files | default([]) }}"
when: security_harden_files | default(false) | bool
```

- Task Set restrictive file permissions for sensitive files:

- **file**: Module này để quản lý tệp và thư mục.
- **path**: Tham số này là đường dẫn đến tệp hoặc thư mục.



- **mode**: Tham số này là cấu hình quyền truy cập của thư mục, mặc định là 0600(SpecialBit/User/Group/Other: rw-----), nghĩa là chỉ chủ sở hữu được đọc hoặc ghi.
- **owner**: Tham số cấu hình chủ sở hữu của thư mục, mặc định là root.
- **group**: Tham số cấu hình nhóm của thư mục, mặc định là root.
- **loop**: Vòng lặp lấy danh sách các tệp cần bảo mật.
- **when**: Chỉ chạy task khi biến `security_harden_files` đặt là true.

2.3.2.3. Handler

```
- name: restart sshd
systemd:
  name: ssh
  state: restarted
ignore_errors: true
```

- Task restart-sshd:
 - **systemd**: Module này để quản lý các dịch vụ ở trên Linux.
 - **name**: Tham số này là tên các dịch vụ cần được thao tác.
 - **state**: Tham số chỉ trạng thái của dịch vụ, ở đây là restarted nghĩa là khởi động lại SSH.
 - **ignore_errors**: Nếu xảy ra lỗi thì Ansible bỏ qua lỗi và tiếp tục chạy.

2.3.3. Database

```
- name: Install MariaDB server and tools
apt:
  name:
    - mariadb-server
    - mariadb-client
    - python3-pymysql
  state: present
  update_cache: true
```

- Task Install MariaDB server and tools:
 - Tương tự như tác vụ tải gói tin về đã được đề cập ở trên, tác vụ này tải 3 gói tin cần thiết cho MariaDB là mariadb-server, mariadb-client, python3-pymysql.

```
- name: Configure MariaDB bind address
lineinfile:
  path: /etc/mysql/mariadb.conf.d/50-server.cnf
  regexp: '^bind-address'
  line: "bind-address = {{ mariadb_bind_address }}"
  backup: yes
```

- Task Configure MariaDB bind address:
 - Task này để cấu hình cho MariaDB, tạo một bản sao lưu trước khi gán giá trị của bind-address thành giá trị của biến `mariadb_bind_address`, nếu biến không tồn tại thì mặc định là “0.0.0.0”.



```
- name: Configure MariaDB port
lineinfile:
  path: /etc/mysql/mariadb.conf.d/50-server.cnf
  regexp: '^port'
  line: "port = {{ mariadb_port }}"
  insertafter: '^\[mysqld\]'
  backup: yes
```

- Task Configure MariaDB port:

- Task này để cấu hình cho MariaDB, tạo một bản sao lưu trước khi gán giá trị của mariadb_port thành giá trị của biến mariadb_port, nếu biến không tồn tại thì mặc định là 3306.

```
- name: Ensure skip-networking disabled
lineinfile:
  path: /etc/mysql/mariadb.conf.d/50-server.cnf
  regexp: '^skip-networking'
  line: 'skip-networking = 0'
  insertafter: '^\[mysqld\]'
  backup: yes
```

- Task Ensure skip-networking disabled:

- Task này để cấu hình cho MariaDB, gán giá trị của skip-networking thành 0.

```
- name: Restart MariaDB
service:
  name: mariadb
  state: restarted
  enabled: true
```

- Task Restart MariaDB:

- **service:** Module quản lý trạng thái dịch vụ.
- **name:** Tham số này chỉ tên dịch vụ cần quản lý.
- **state:** Tham số này chỉ trạng thái của dịch vụ, ở đây đặt là restarted, dịch vụ khởi động lại để áp dụng các thay đổi trên.
- **enable:** Tham số này đảm bảo dịch vụ tự động khởi động khi reboot.

```
- name: Wait for MariaDB to listen on TCP
wait_for:
  host: 127.0.0.1
  port: "{{ mariadb_port | int }}"
  delay: 1
  timeout: 30
```

- Task Restart MariaDB:

- **wait_for:** Module kiểm tra và chờ đợi một điều kiện được đáp ứng.
- **host:** Tham số này chỉ định địa chỉ IP được kiểm tra.



- **port**: Tham số này chỉ định cổng kiểm tra là của mariadb(3306) được khai báo trong file db.yml, bộ lọc int để đảm bảo giá trị của cổng là số nguyên.
- **delay**: Tham số này là thời gian chờ trước khi bắt đầu kiểm tra, ở đây là 1 giây.
- **timeout**: Tham số này là thời gian chờ tối đa để kiểm tra lại, ở đây là 30 giây.

```
- name: Ensure database exists
community.mysql.mysql_db:
  name: "{{ db_name }}"
  state: present
  login_unix_socket: /var/run/mysqld/mysqld.sock
```

- Task Ensure database exists:

- **community.mysql.mysql_db**: Module được dùng để quản lý cơ sở dữ liệu.
- **name**: Tên cơ sở dữ liệu được khai báo trong file all.yml.
- **state**: Tham số này chỉ trạng thái của cơ sở dữ liệu, đảm bảo nó tồn tại.
- **login_unix_socket**: Sử dụng socket UNIX cục bộ để đăng nhập với quyền root của hệ thống để thiết lập cơ sở dữ liệu.

```
- name: Ensure DB user exists with privileges
community.mysql.mysql_user:
  name: "{{ db_user }}"
  password: "{{ db_password }}"
  priv: "{{ db_name }}.*:ALL"
  host: "%"
  state: present
  login_unix_socket: /var/run/mysqld/mysqld.sock
```

- Task Ensure database exists:

- **name**: Tên người dùng cơ sở dữ liệu được khai báo trong file all.yml.
- **password**: Mật khẩu người dùng cơ sở dữ liệu cần tạo được khai báo trong file all.yml.
- **priv**: Tham số này để cấp quyền, cho phép người dùng này có tất cả (ALL) quyền trên tất cả các bảng (*) trong cơ sở dữ liệu db_name.
- **host**: Tham số cho phép người dùng này kết nối từ bất kỳ host nào (%).
- **state**: Tham số này đảm bảo người dùng này tồn tại(present).
- **login_unix_socket**: Dùng lại phương pháp xác thực socket để tạo người dùng và cấp quyền.

2.3.4. Haproxy

2.3.4.1. Tasks/main.yml

```
- name: Install HAProxy
  apt:
    name: haproxy
    state: present
    update_cache: true
```

- Task Install MariaDB server and tools:
 - o Tương tự như tác vụ tải gói tin về đã được đề cập ở trên, tác vụ này tải gói tin cần thiết cho Haproxy là haproxy.

```
- name: Backup default HAProxy config
  copy:
    src: /etc/haproxy/haproxy.cfg
    dest: /etc/haproxy/haproxy.cfg.backup
    remote_src: true
  when: ansible_os_family == "Debian"
  ignore_errors: true
```

- Task Backup default HAProxy config:
 - o **copy**: Module để sao chép file.
 - o **src**: Tham số chỉ đường dẫn của file nguồn trên Managed Node.
 - o **dest**: Tham số chỉ đường dẫn của file được sao lưu.
 - o **remote_src**: Tham số chỉ nguồn nằm trên Managed Node (không phải Control Node).
 - o **when**: Điều kiện là chỉ chạy trên hệ thống Debian/Ubuntu.
 - o **ignore_errors**: Bỏ qua nếu xảy ra lỗi và tiếp tục chạy.

```
- name: Deploy HAProxy configuration
  template:
    src: haproxy.cfg.j2
    dest: /etc/haproxy/haproxy.cfg
    owner: root
    group: root
    mode: '0644'
  notify: restart haproxy
```

- Task Deploy HAProxy configuration:
 - o **template**: Module này sử dụng Jinja2 template.
 - o **src**: Tham số này chỉ đường dẫn của template trong roles/haproxy/templates/.
 - o **dest**: Tham số này chỉ đường dẫn của file cấu hình.
 - o **owner**: Tham số này chỉ chủ sở hữu file là root.
 - o **group**: Tham số này chỉ nhóm file là root
 - o **mode**: Tham số này đảm bảo ai cũng có thể đọc tệp này nhưng chỉ có chủ sở hữu mới có thể thay đổi (0644: rw-r--r--).



- **notify**: Đây là một notify để kích hoạt handler khởi động lại haproxy khi có thay đổi.

```
- name: Start HAProxy service
  service:
    name: haproxy
    state: started
    ignore_errors: true
```

- Task Start HAProxy service:

- Tương tự như MariaDB, sau khi kết thúc cấu hình thì Task này có nhiệm vụ khởi động Haproxy.

```
- name: Check HAProxy process
  command: pgrep haproxy || echo "0"
  register: haproxy_check
  changed_when: false
  failed_when: false
```

- Task Check HAProxy process:

- **command**: Tham số này để chạy lệnh kiểm tra tiến trình, nếu không tìm thấy thì in ra 0.
- **register**: Tham số này để lưu kết quả vào biến haproxy_check.
- **changed_when**: Tham số này để không đánh dấu task là "changed", vì kiểm tra trạng thái không được xem là thay đổi cấu hình.
- **failed_when**: Tham số này để không đánh dấu task là "failed" dù lệnh trả về lỗi, điều này cần thiết để đảm bảo Playbook không bị dừng nếu HAProxy không chạy.

```
- name: Display HAProxy status
  debug:
    msg: "HAProxy process check: {{ haproxy_check.stdout }}"
```

- Task Display HAProxy status:

- **debug**: Module này để in thông tin ra console.
- **msg**: Thông số này để in kết quả từ Task kiểm tra Haproxy ở trên.

2.3.4.2. Handlers/main.yml

```
- name: restart haproxy
  service:
    name: haproxy
    state: restarted
    ignore_errors: true
```

- Task restart haproxy:

- Tương tự như MariaDB, sau khi cấu hình xong thì cần khởi động lại để áp dụng các thay đổi.

2.3.4.3. *Templates/haproxy.cfg.j2*

- Đây là tệp cấu hình hoàn chỉnh cho haproxy:

```
global
    log /dev/log      local0
    log /dev/log      local1 notice
    chroot /var/lib/haproxy
    stats socket /run/haproxy/admin.sock mode 660 level admin
    stats timeout 30s
    user haproxy
    group haproxy
    daemon
```

- Đây là phần cấu hình toàn cục, nơi định nghĩa các tham số và an ninh cho HAProxy:

- **log**: Đầu tiên là gửi thông báo nhật ký đến máy chủ hệ (dev/log) thông qua các cơ sở local0 và local1. local1 thì ở mức độ notice, được dùng để ghi các sự kiện quan trọng, còn local0 không để mức độ thì chỉ ghi các lỗi và cảnh báo.
- **chroot**: Thiết lập môi trường để giới hạn truy cập của tiến trình HAProxy chỉ trong thư mục được chỉ định, giúp tăng cường độ bảo mật.
- **stats**: Dùng với socket để tạo một socket UNIX cho phép quản trị viên hoặc các script quản lý kiểm tra và điều khiển HAProxy qua giao diện dòng lệnh với quyền 660(rw-rw----). Dùng với timeout để giới hạn thời gian kết nối là 30 giây.
- **user**: Chạy với quyền của user haproxy.
- **group**: Chạy với quyền của nhóm haproxy.
- **daemon**: Yêu cầu haproxy chạy ở chế độ nền.

```
defaults
    log      global
    mode     http
    option   httplog
    option   dontlognull
    timeout  connect 5000ms
    timeout  client  50000ms
    timeout  server  50000ms
    errorfile 400 /etc/haproxy/errors/400.http
    errorfile 403 /etc/haproxy/errors/403.http
    errorfile 408 /etc/haproxy/errors/408.http
    errorfile 500 /etc/haproxy/errors/500.http
    errorfile 502 /etc/haproxy/errors/502.http
    errorfile 503 /etc/haproxy/errors/503.http
    errorfile 504 /etc/haproxy/errors/504.http
```

- Đây là phần cấu hình mặc định:

- **log**: Sử dụng cấu hình nhật ký đã được định nghĩa trong phần global.
- **mode**: Đặt chế độ hoạt động mặc định là HTTP, cho phép HAProxy phân tích lưu lượng HTTP.



- **option:** httplog giúp kích hoạt việc ghi nhật ký chi tiết các yêu cầu HTTP, dontlognull giúp ngăn việc ghi nhật ký các kết nối không tạo ra bất kỳ yêu cầu nào.
- **timeout:** Thời gian tối đa để thiết lập kết nối từ HAProxy đến một máy chủ backend(5 giây), client và server(50 giây).
- **errorfile:** Cấu hình HAProxy cho các tệp lỗi HTML khi nó phát hiện lỗi cụ thể.

```
frontend http_front
  bind *:80
  default_backend web_backend
```

- Frontend định nghĩa các cổng mạng mà HAProxy lắng nghe:
 - **frontend:** Tên của frontend.
 - **bind:** Yêu cầu HAProxy lắng nghe tất cả các giao diện mạng (*) trên cổng 80.
 - **default_backend web_backend:** Chỉ định rằng tất cả lưu lượng truy cập nhận được trên frontend này được chuyển đến web_backend.

```
backend web_backend
  balance roundrobin
  option httpchk GET /
  http-check expect status 200 301 302
  server web1 {{ haproxy_backend_web1 |
default(hostvars['web1']['ansible_host'] + ':80') }} check
  server web2 {{ haproxy_backend_web2 |
default(hostvars['web2']['ansible_host'] + ':80') }} check
```

- Backend định nghĩa nhóm các máy chủ web và cách lưu:
 - **backend:** Tên của nhóm máy chủ web.
 - **balance:** Thuật toán cân bằng tải. Round-robin luân phiên chuyển tiếp các yêu cầu đến từng máy chủ backend.
 - **option:** Kích hoạt kiểm tra bằng cách gửi một yêu cầu HTTP GET đến đường dẫn gốc của mỗi máy chủ backend.
 - **http-check:** Chỉ định rằng máy chủ backend được coi là vẫn hoạt động nếu nó phản hồi với mã trạng thái 200 (OK), 301 (Moved Permanently) hoặc 302 (Found).
 - **server:** Định nghĩa các máy chủ backend(web1 và web2) bằng các biến Ansible để xác định địa chỉ IP và cổng.
 - **check:** Kích hoạt việc kiểm tra liên tục theo cấu hình httpchk và http-check expect.

```
stats enable
stats uri /stats
stats refresh 30s
stats auth admin:{{ haproxy_stats_password }}
```

- Phần cuối là cấu hình trang thống kê giám sát của HAProxy.
 - **stats: enable** là bật tính năng thống kê, uri để thiết lập đường dẫn URL để truy cập trang thống kê, refresh để trang thống kê tự động làm mới sau mỗi 30 giây,



auth là để bảo vệ trang thống kê bằng yêu cầu xác thực. Tên người dùng là admin và mật khẩu được lấy từ haproxy_stats_password.

2.3.5. Redis

2.3.5.1. Redis/Tasks/main.yml

```
- name: Install Redis server
apt:
name: redis-server
state: present
update_cache: true
```

- Task Install Redis server:

- Tương tự như tác vụ tải gói tin về đã được đề cập ở trên, tác vụ này tải gói tin cần thiết cho Redis là redis-server.

```
- name: Configure Redis bind address
lineinfile:
path: /etc/redis/redis.conf
regexp: '^bind '
line: "bind {{ redis_bind_address }}"
backup: yes
```

- Task Configure Redis bind address:

- Task này để cấu hình cho Redis, tạo một bản sao lưu trước khi gán giá trị của bind thành giá trị của biến redis_bind_address, mặc định là "0.0.0.0".

```
- name: Configure Redis port
lineinfile:
path: /etc/redis/redis.conf
regexp: '^port '
line: "port {{ redis_tcp_port }}"
backup: yes
```

- Task Configure Redis port:

- Task này để cấu hình cho Redis, tạo một bản sao lưu trước khi gán giá trị của port thành giá trị của biến redis_tcp_port, mặc định là 6379.

```
- name: Restart Redis
service:
name: redis-server
state: restarted
enabled: true
```

- Task Restart Redis:

- Sau khi cấu hình hoàn tất thì Task này để Restart dịch vụ Redis để áp dụng những thay đổi.

2.3.6. Web

2.3.6.1. *tasks/main.yml*

```
- name Install Apache and PHP packages
  apt
    name "{{ ['apache2'] + php_packages }}"
    state present
    update_cache true
```

- Task Install Apache and PHP packages:
 - o Tương tự như tác vụ tải gói tin về đã được đề cập ở trên, tác vụ này tải gói tin cần thiết cho Apache và PHP là apache2 và danh sách gói php trong php_packages.

```
- name Ensure app root exists
  file
    path "{{ web_app_root }}"
    state directory
    owner www-data
    group www-data
    mode '0755'
```

- Task Ensure app root exists:
 - o Task này đảm bảo đường dẫn đến thư mục gốc tồn tại, thiết lập chủ sở hữu và nhóm sở hữu là www-data, đặt quyền truy cập là 0755(rwer-er-e).

```
- name Configure Apache listen port
  lineinfile
    path /etc/apache2/ports.conf
    regexp '^Listen '
    line "Listen {{ apache_listen_port }}"
    create yes
```

- Task Configure Apache listen port:
 - o Task này để cấu hình cho Apache, nếu tệp không tồn tại thì tự tạo tệp, gán giá trị của Listen thành giá trị của biến apache_listen_port.

```
- name Deploy VirtualHost
  template
    src vhost.conf.j2
    dest /etc/apache2/sites-available/app.conf
    mode '0644'
```

- Task Deploy VirtualHost:
 - o Task này để triển khai tệp cấu hình có sẵn vhost.conf.j2, hình có sẵn tạo tệp cấu hình ở Managed Node với quyền truy cập là 0644(rw-r--r--)



```
- name Disable default site
file
  path /etc/apache2/sites-enabled/000-default.conf
  state absent
```

- Task Disable default site:

- Task này để đảm bảo tệp cấu hình mặc định 000-default.conf không tồn tại.

```
- name Enable required modules
command a2enmod rewrite
register enmod
failed_when false
changed_when enmod.rc == 0 and (enmod.stdout is not defined
or 'already enabled' not in enmod.stdout)
```

- Task Enable required modules:

- Task này thực thi lệnh `a2enmod rewrite` để kích hoạt module `mod_rewrite`, cho phép máy chủ web viết lại URL giúp dễ nhìn và dễ cấu hình. Lưu kết quả vào biến `enmod`. Ngăn task báo lỗi và dừng Playbook nếu module đã được bật (`a2enmod` trả về lỗi nếu đã được bật), chỉ đánh dấu Task là `changed` nếu lệnh thành công và kết quả không chứa chuỗi `already enabled`.

```
- name Enable app site
command a2ensite app
register ens
failed_when false
changed_when ens.rc == 0 and (ens.stdout is not defined or
'already enabled' not in ens.stdout)
```

- Task Enable app site:

- Task này thực hiện tương tự như task ở phía trên, thực thi lệnh `a2ensite app` để kích hoạt Virtual Host.

```
- name Create index.php sample
copy
  dest "{{ web_app_root }}/index.php"
  content |
    <?php
    phpinfo();
  owner www-data
  group www-data
  mode '0644'
```

- Task Create index.php sample:

- Task này để tạo tệp `index.php` ở trên Managed Node với nội dung là trang web php mẫu, đặt quyền sở hữu của chủ và nhóm là `www-data`, quyền truy cập là `0644(rw-r--r--)`

```
- name Restart apache
  service
    name apache2
    state restarted
    enabled true
```

- Task Restart Apache:

- Sau khi cấu hình hoàn tất thì Task này để Restart dịch vụ Apache để áp dụng những thay đổi.

2.3.6.2. *xtemplates/vhost.conf.j2*

- Tập này định nghĩa một cấu hình máy chủ ảo riêng biệt trên Apache, sử dụng các biến Ansible để cấu hình:

```
<VirtualHost *:{{ apache_listen_port }}>
  ServerName {{ web_server_name }}
  DocumentRoot {{ web_app_root }}

  <Directory {{ web_app_root }}>
    AllowOverride All
    Require all granted
  </Directory>

  ErrorLog ${APACHE_LOG_DIR}/app_error.log
  CustomLog ${APACHE_LOG_DIR}/app_access.log combined
</VirtualHost>
```

- **VirtualHost**: Đây là đánh dấu chỗ bắt đầu một Virtual Host. * nghĩa là nó lắng nghe trên tất cả các địa chỉ IP có sẵn trên máy chủ. Port được lấy từ biến `apache_listen_port`.

- **ServerName**: Đặt tên máy chủ cho Virtual Host bằng biến `web_server_name`.
- **DocumentRoot**: Chỉ định thư mục gốc của web được khai báo trong file `all.yml`.

- **Directory**: Chỉnh sửa cấu hình cho thư mục trong biến `web_app_root`.

- **AllowOverride All**: Cho phép sử dụng các tệp `.htaccess` bên trong thư mục này để cấu hình lại URL hoặc thêm các quy tắc bảo mật.
- **Require all granted**: Đây là cú pháp kiểm soát truy cập của Apache. Nó cấp quyền truy cập cho tất cả mọi người vào thư mục này.

- Cấu hình nhật ký:

- **ErrorLog**: Chỉ định tệp nhật ký nơi Apache ghi lại các thông báo lỗi hoặc các vấn đề khác. Biến `${APACHE_LOG_DIR}` là đường dẫn thư mục log mặc định của Apache
- **CustomLog**: Chỉ định tệp nhật ký nơi Apache ghi lại các yêu cầu truy cập thành công. Format `combined` là một định dạng nhật ký bao gồm địa chỉ IP, thời gian, phương thức HTTP, mã trạng thái, và kích thước phản hồi.

2.3.7. Wordpress

2.3.7.1. *tasks/main.yml*

```
- name: Install unzip and curl for WordPress deployment
  apt:
    name:
      - unzip
      - curl
    state: present
    update_cache: true
```

- Task Install unzip and curl for WordPress deployment:

- Tương tự như tác vụ tải gói tin về đã được đề cập ở trên, tác vụ này tải các gói tin cần thiết là unzip và curl.

```
- name: Ensure app root exists
  file:
    path: "{{ web_app_root }}"
    state: directory
    owner: www-data
    group: www-data
    mode: '0755'
```

- Task Ensure app root exists:

- Task này đảm bảo đường dẫn đến thư mục gốc tồn tại, thiết lập chủ sở hữu và nhóm sở hữu là www-data, đặt quyền truy cập là 0755(rwer-er-e).

```
- name: Download WordPress
  get_url:
    url: https://wordpress.org/latest.zip
    dest: /tmp/wordpress.zip
    mode: '0644'
  when: wordpress_install | default(true)
```

- Task Download WordPress:

- **get_url**: Module dùng để tải tệp từ xa.
- **url**: Tham số chỉ URL để tải về phiên bản WordPress mới nhất.
- **dest**: Tham số chỉ đường dẫn để lưu tệp đã tải xuống vào thư mục tạm thời /tmp/ trên Managed Node.
- **when**: Điều kiện mặc định là true.

```
- name: Unarchive WordPress into app root
unarchive:
  src: /tmp/wordpress.zip
  dest: "{{ web_app_root }}"
  remote_src: true
  creates: "{{ web_app_root }}/wordpress"
when: wordpress_install | default(true)
```

- Task Unarchive WordPress into app root:

- **unarchive:** Module dùng để giải nén tệp lưu trữ.
- **src:** Tham số chỉ đường dẫn tệp nguồn cần giải nén.
- **dest:** Tham số chỉ đường dẫn thư mục đích giải nén được định nghĩa bằng biến `web_app_root`.
- **remote_src:** Tham số chỉ ra rằng tệp nguồn nằm trên Managed Node, không phải Control Node.
- **creates:** Tham số để chỉ nếu đường dẫn chưa tồn tại, tự tạo một đường dẫn mới.
- **when:** Điều kiện mặc định là true.

```
- name: Move WordPress files to app root
command:
  cmd: bash -lc "shopt -s dotglob && mv -f {{ web_app_root }}
  }}/wordpress/* {{ web_app_root }}/ && rmdir {{ web_app_root }}
  }}/wordpress"
  chdir: "/"
  creates: "{{ web_app_root }}/wp-settings.php"
when: wordpress_install | default(true)
```

- Task Move WordPress files to app root:

- **command:** Module chạy lệnh shell.
- **cmd:** Lệnh shell đầu tiên thực hiện giải nén tệp .zip của WordPress ra thành `/var/www/app/wordpress/`. Lệnh này di chuyển tất cả nội dung từ thư mục `/wordpress/` lên một cấp (`/var/www/app/`). Cuối cùng, xóa thư mục `/wordpress` rỗng.
- **creates:** Task chỉ chạy nếu tệp `wp-settings.php` chưa được di chuyển vào thư mục gốc.
- **when:** Điều kiện mặc định là true.

```
- name: Generate wp-config.php
  template:
    src: wp-config.php.j2
    dest: "{{ web_app_root }}/wp-config.php"
    mode: '0644'
    owner: www-data
    group: www-data
```

- Task Generate wp-config.php:
 - o Task này để cấu hình file wp-config.php trên Managed Node với các biến Ansible để kết nối đến cơ sở dữ liệu.

```
- name: Ensure ownership for web root
  file:
    path: "{{ web_app_root }}"
    state: directory
    owner: www-data
    group: www-data
    recurse: true
  when: wordpress_set_owner | default(true)
```

- Task Ensure ownership for web root:
 - o Task này để đảm bảo quyền sở hữu của thư mục gốc (kể cả các thư mục con trong đó) đều là của www-data, điều kiện mặc định là true.

2.3.7.2. *templates/wp-config.php.j2*

```
<?php
// Basic WordPress configuration generated by Ansible

define( 'DB_NAME', '{{ db_name }}' );
define( 'DB_USER', '{{ db_user }}' );
define( 'DB_PASSWORD', '{{ db_password }}' );
define( 'DB_HOST', '{{ wordpress_db_host }}' );
define( 'DB_CHARSET', 'utf8' );
define( 'DB_COLLATE', '' );
```

- Cấu hình để kết nối cơ sở dữ liệu: cấu hình các dữ liệu cơ bản như tên cơ sở dữ liệu, tên người dùng, mật khẩu, địa chỉ máy chủ, bộ ký tự và bộ đối chiếu.

```
// Keys and salts (basic random fallback if not provided)
{% set salts = wordpress_auth_salts | default('') %}
{% if salts %}
{{ salts | trim }}
{% else %}
define('AUTH_KEY',          'ansible-{{ 999999999 | random |
string }}');
define('SECURE_AUTH_KEY',    'ansible-{{ 999999999 | random |
string }}');
define('LOGGED_IN_KEY',      'ansible-{{ 999999999 | random |
string }}');
define('NONCE_KEY',          'ansible-{{ 999999999 | random |
string }}');
define('AUTH_SALT',          'ansible-{{ 999999999 | random |
string }}');
define('SECURE_AUTH_SALT',   'ansible-{{ 999999999 | random |
string }}');
define('LOGGED_IN_SALT',     'ansible-{{ 999999999 | random |
string }}');
define('NONCE_SALT',         'ansible-{{ 999999999 | random |
string }}');
{% endif %}
```

- Thiết lập các khóa và Salt bảo mật:

- Chạy nếu biến `wordpress_auth_salts` không được định nghĩa.
- Tạo ra 8 dòng `define` với các giá trị ngẫu nhiên để tạo ra khóa ngẫu nhiên.

```
$table_prefix = 'wp_';
define( 'WP_DEBUG', false );

if ( ! defined( 'ABSPATH' ) ) {
    define( 'ABSPATH', __DIR__ . '/' );
}

require_once ABSPATH . 'wp-settings.php';
```

- Cấu hình chung:

- **table_prefix**: Bảng tiền tố trong cơ sở dữ liệu. Giúp phân biệt các bộ cài đặt WordPress khác nhau trong cùng một cơ sở dữ liệu.
- **WP_DEBUG**: Vô hiệu hóa chế độ debug của WordPress để tăng tính bảo mật.
- **ABSPATH**: Định nghĩa hằng số cho đường dẫn tuyệt đối nếu nó chưa được định nghĩa.
- **require_once**: Khởi động toàn bộ hệ thống WordPress bằng cách thực thi file `wp-settings.php`.

2.3.8. Backup

2.3.8.1. *tasks/main.yml*

```
- name: Ensure backup directory exists
  file:
    path: "{{ backup_dir }}"
    state: directory
    owner: root
    group: root
    mode: '0755'
```

- Task Ensure back directory exists:

- Task này đảm bảo đường dẫn đến thư mục back up tồn tại, thiết lập chủ sở hữu và nhóm sở hữu là root, đặt quyền truy cập là 0755(rwer-er-e).

```
- name: Install backup tools
  apt:
    name:
      - rsync
      - gzip
      - cron
    state: present
    update_cache: true
```

- Task Install backup tools:

- Task này tải các gói tin cần thiết cho việc backup bao gồm rsync, gzip và cron.

```
- name: Ensure cron service running
  service:
    name: cron
    state: started
    enabled: true
```

- Task Ensure cron service running:

- Task này đảm bảo dịch vụ cron đang chạy và dịch vụ vẫn chạy sau khi reboot.

```
- name: Create backup script
copy:
  dest: /usr/local/bin/backup_app.sh
  mode: '0755'
  content: |
    #!/usr/bin/env bash
    set -euo pipefail
    TS=$(date +"%Y%m%d-%H%M%S")
    mkdir -p {{ backup_dir }}
    # Web files backup (from web servers via rsync or local)
    {% if backup_web_hosts is defined and backup_web_hosts |
length > 0 %}
    {% for host in backup_web_hosts %}
    rsync -avz --delete {{ host }}:{{ web_app_root }}/ {{
backup_dir }}/web-{{ loop.index }}-$TS/ || true
    tar -czf {{ backup_dir }}/app-files-{{ loop.index }}-
$TS.tgz -C {{ backup_dir }}/web-{{ loop.index }}-$TS . || true
    rm -rf {{ backup_dir }}/web-{{ loop.index }}-$TS || true
    {% endfor %}
    {% else %}
    # Local web files backup (if web_app_root exists locally)
    if [ -d "{{ web_app_root }}" ]; then
    tar -czf {{ backup_dir }}/app-files-$TS.tgz -C {{
web_app_root }} . || true
    fi
    {% endif %}
    # DB dump (local or remote)
    {% if backup_db_host is defined and backup_db_host != ''
%}

    # Remote DB backup
    if command -v mysqldump >/dev/null 2>&1; then
    mysqldump -h {{ backup_db_host.split(':')[0] }} -P {{
backup_db_host.split(':')[1] | default('3306') }} -u {{
backup_db_user | default('root') }} -p'{{ backup_db_password |
default('') }}' --all-databases > {{ backup_dir }}/db-$TS.sql ||
true

    gzip -f {{ backup_dir }}/db-$TS.sql || true
    fi
    {% else %}
    # Local DB backup (if mysql available)
    if command -v mysqldump >/dev/null 2>&1; then
    mysqldump -u root --protocol=socket --all-databases > {{
backup_dir }}/db-$TS.sql || true
    gzip -f {{ backup_dir }}/db-$TS.sql || true
    fi
    {% endif %}
```



- Task Create backup script:
 - o **set**: Đây là lệnh cài đặt an toàn khi để -euo pipefail để thoát ngay khi có bất kỳ lệnh nào thất bại.
 - o **TS**: Tạo định dạng thời gian theo Y/m/d/H/M/S.
 - o **mkdir**: Đảm bảo thư mục lưu trữ sao lưu tồn tại, nếu chưa thì tạo mới.
- Sao lưu tệp web:
 - o **rsync**: Lặp lại qua từng host web trong backup_web_hosts để sao lưu từng máy chủ. Sử dụng với các flag -avz để sao lưu ở chế độ Archive và nén khi truyền, --delete để xóa tệp trên đích đến nếu chúng không còn tồn tại trên nguồn, || true nếu lệnh sao lưu thất bại, script bỏ qua lỗi này. Nếu thư mục gốc tồn tại cục bộ, sao lưu ở cục bộ.
 - o **tar**: Nén thư mục thành một tệp .tgz .
 - o **rm**: Xóa thư mục tạm thời sau khi nén.
- Sao lưu cơ sở dữ liệu:
 - o **mysqldump**: Thực hiện dump DB từ xa, sao lưu tất cả cơ sở dữ liệu trên host lấy từ biến backup_db_host. Nếu cơ sở dữ liệu tồn tại ở cục bộ thì sao lưu cục bộ.
 - o **gzip**: Nén tệp SQL đã tạo.

```
- name: Configure cron job
cron:
  name: "App backup"
  user: root
  minute: "{{ backup_time.split(' ')[0] }}"
  hour: "{{ backup_time.split(' ')[1] }}"
  day: "{{ backup_time.split(' ')[2] }}"
  month: "{{ backup_time.split(' ')[3] }}"
  weekday: "{{ backup_time.split(' ')[4] }}"
  job: "/usr/local/bin/backup_app.sh"
when: backup_cron_enabled | bool
```

- Task Configure cron job:
 - o Task này cấu hình lịch chạy cho cron, như đã đề cập trong file backup.yml, cron chạy vào 3 giờ sáng mỗi ngày.

```
- name: Verify cron job exists
shell: crontab -l -u root | grep -c "backup_app.sh" || echo
"0"
register: cron_check
changed_when: false
failed_when: false
```

- Task Verify cron job exists:
 - o Task này đảm bảo lịch đã lập cho cron đã được thêm vào crontab của root chưa, không đánh dấu là thay đổi vì đây là hành động kiểm tra.



```
- name: Display cron validation
  debug:
    msg: "Cron job validation: {{ cron_check.stdout }} match(es)
found"
  when: cron_check.stdout | int > 0
```

- Task Display cron validation:
 - o Task này hiển thị kết quả kiểm tra của cron job.

```
- name: Test backup script manually
  shell: /usr/local/bin/backup_app.sh
  register: backup_test
  changed_when: false
  failed_when: false
  when: backup_test_enabled | default(false) | bool
```

- Task Test backup script manually:
 - o Task này chạy script sao lưu để kiểm tra chức năng ngay sau khi triển khai.

```
- name: Check backup directory contents
  find:
    paths: "{{ backup_dir }}"
    patterns: "*.tgz,*.sql.gz"
  register: backup_files
  changed_when: false
```

- Task Check backup directory contents:
 - o Task này tìm kiếm các tệp sao lưu mới được tạo trong thư mục backup_dir.

```
- name: Display backup files found
  debug:
    msg: "Found {{ backup_files.files | length }} backup file(s)
in {{ backup_dir }}"
```

- Task Display backup files found:
 - o Task này Hiển thị số lượng tệp sao lưu được tìm thấy, xác nhận rằng quá trình sao lưu đã hoạt động.

2.3.9. Playbooks

```
- name: Common base setup
hosts: all
become: true
roles:
  - role: common

- name: Web servers
hosts: web
become: true
roles:
  - role: web
  - role: wordpress

- name: Database servers
hosts: db
become: true
roles:
  - role: db

- name: Redis servers
hosts: redis
become: true
roles:
  - role: redis

- name: Load Balancer
hosts: haproxy
become: true
roles:
  - role: haproxy

- name: Backup nodes
hosts: backup
become: true
roles:
  - role: backup
```

- Đây là một Playbook hoàn chỉnh, nó thực thi các role đã được khai báo ở trên:
 - o **hosts**: Chỉ định nhóm máy chủ sẽ thực thi task, lúc này việc chia host ra thành nhiều nhóm phát huy khả năng của nó.
 - o **become**: Cho phép Ansible leo thang đặc quyền lên thành root để thực thi tác vụ.
 - o **roles**: Đây là danh sách các vai trò mà Ansible sẽ áp dụng cho task.

3. KẾT QUẢ VÀ KẾT LUẬN

3.1. Kết quả

```
$ docker compose up
[+] Running 7/7
✓ Network docker_default      Created
✓ Container ansible_backup    Created
✓ Container ansible_db        Created
✓ Container ansible_web2      Created
✓ Container ansible_redis     Created
✓ Container ansible_web        Created
✓ Container ansible_haproxy   Created
Attaching to ansible_backup, ansible_db, ansible_haproxy, ansible_redis, ansible_web, ansible_web2
ansible_db | Server listening on 0.0.0.0 port 22.
ansible_db | Server listening on :: port 22.

ansible_web | Server listening on 0.0.0.0 port 22.
ansible_web2 | Server listening on 0.0.0.0 port 22.

ansible_web | Server listening on :: port 22.

ansible_web2 | Server listening on :: port 22.
ansible_backup | Server listening on 0.0.0.0 port 22.
ansible_backup | Server listening on :: port 22.
ansible_redis | Server listening on 0.0.0.0 port 22.
ansible_redis | Server listening on :: port 22.
ansible_haproxy | Server listening on 0.0.0.0 port 22.
ansible_haproxy | Server listening on :: port 22.
```

Hình 3.1.1. Kết quả chạy lệnh docker compose up

[illegible]

Hình 3.1.2. Chi tiết cấu hình php

```
hope@DESKTOP-PS01DB3:/mnt/d/05-Project/QuanTriMang/code/ansible$ export ANSIBLE_ROLES_PATH=$PWD/roles
ansible-playbook -i inventories/docker/hosts.ini playbooks/site.yml --limit db
[WARNING]: Ansible is being run in a world writable directory
(/mnt/d/05-Project/QuanTriMang/code/ansible), ignoring it as an ansible.cfg source. For more information
see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir

PLAY [Common base setup] *****

TASK [Gathering Facts] *****
ok: [db1]

TASK [common : Update apt cache] *****
ok: [db1]

TASK [common : Install base packages] *****
ok: [db1]

PLAY [Web servers] *****
skipping: no hosts matched

PLAY [Database servers] *****

TASK [Gathering Facts] *****
ok: [db1]

TASK [db : Install MariaDB server and tools] *****
ok: [db1]

TASK [db : Configure MariaDB bind address] *****
ok: [db1]
```

Hình 3.1.3. Kết quả khởi động playbook

```
hope@DESKTOP-PS01DB3:/mnt/d/05-Project/QuanTriMang/code/ansible$ mysql -h 127.0.0.1 -P 3307 -u appuser -p'appsecret!'
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 36
Server version: 10.6.22-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

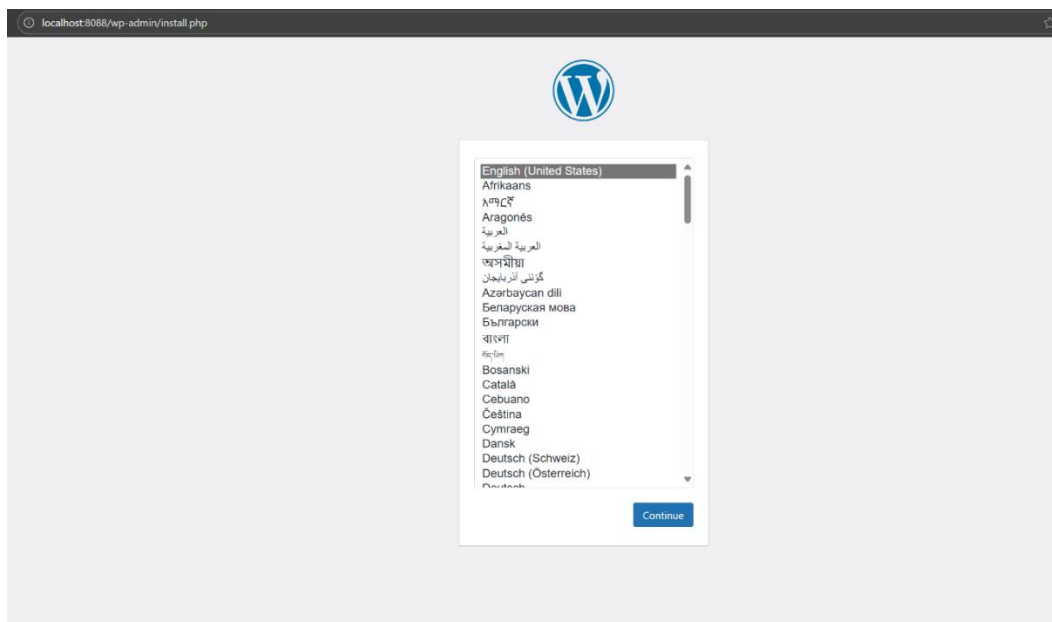
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| appdb    |
| information_schema |
+-----+
2 rows in set (0.001 sec)

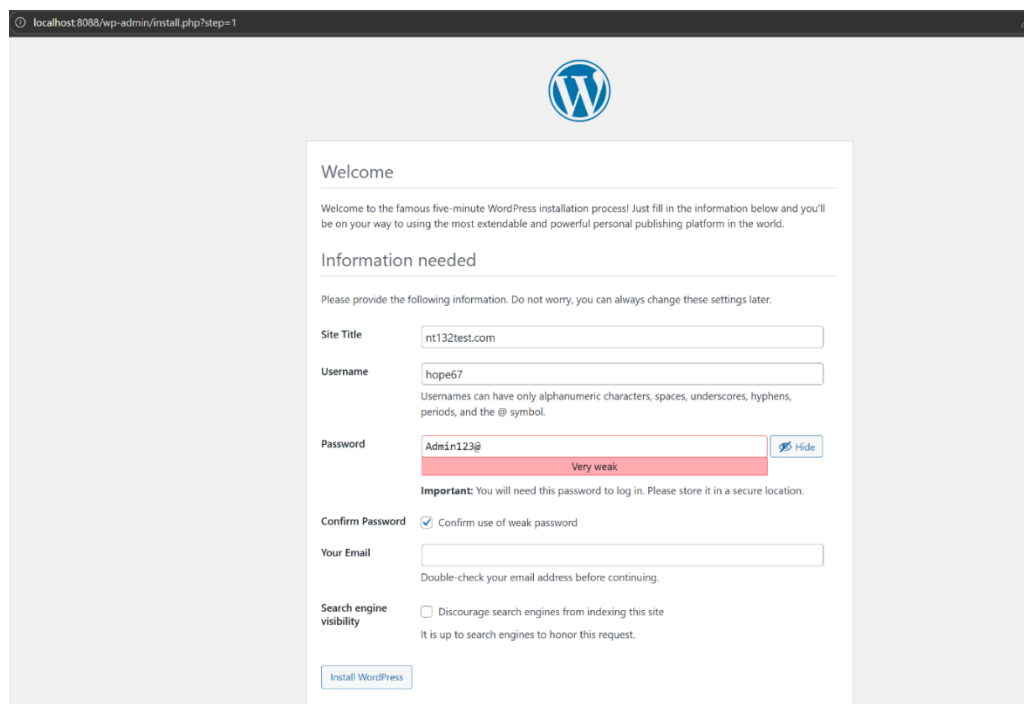
MariaDB [(none)]> SHOW GRANTS FOR 'appuser'@'%';
+-----+
| Grants for appuser@% |
+-----+
| GRANT USAGE ON *.* TO `appuser`@`%` IDENTIFIED BY PASSWORD '*0F2D9F5C5B929AA66050928959D641E432FEB248' |
| GRANT ALL PRIVILEGES ON `appdb`.* TO `appuser`@`%` |
+-----+
2 rows in set (0.001 sec)

MariaDB [(none)]> quit
Bye
```

Hình 3.1.4. Kết quả khởi động database



Hình 3.1.5. Trang WordPress sau khi build



Hình 3.1.6. Đăng kí tài khoản (1)



Welcome

You must provide an email address.

Site Title: nt132test.com

Username: hope67

Password: Admin123@ (Very weak)

Confirm Password: ☒ Confirm use of weak password

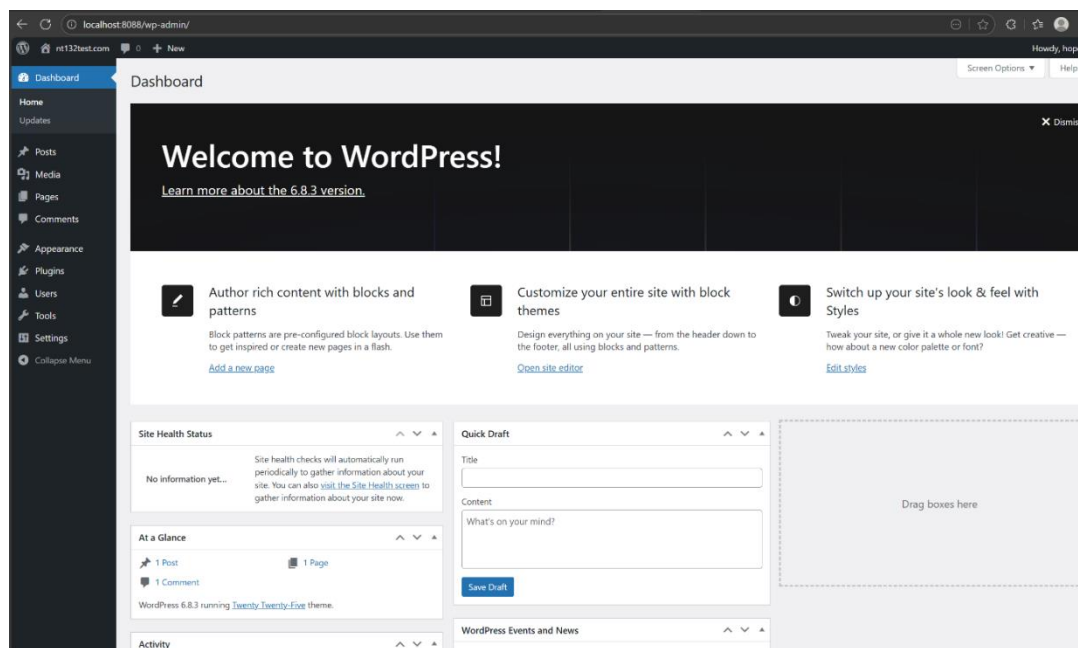
Your Email: vuongthanhdat67@gmail.com

Search engine visibility: ☐ Discourage search engines from indexing this site

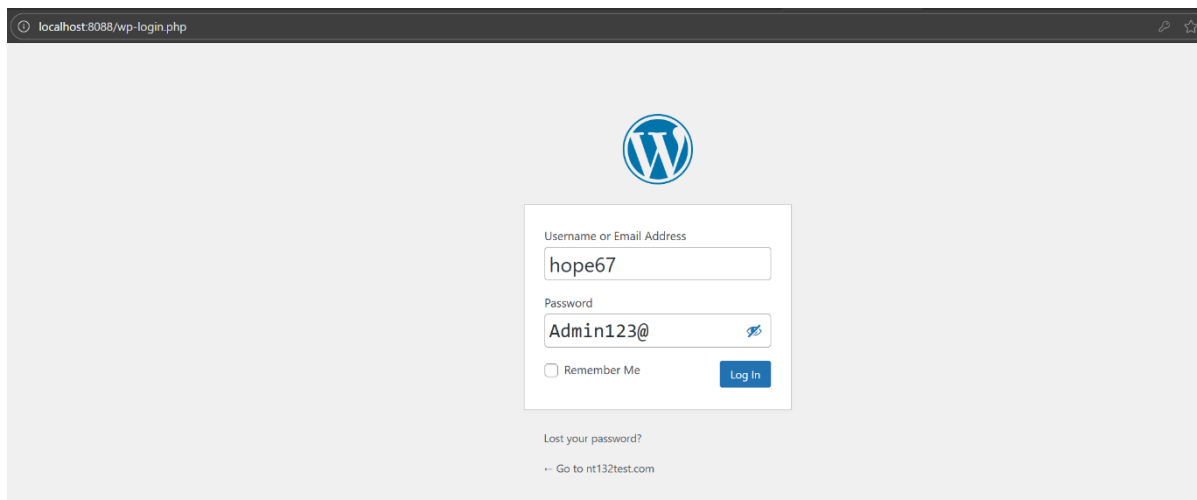
Important: You will need this password to log in. Please store it in a secure location.

Install WordPress

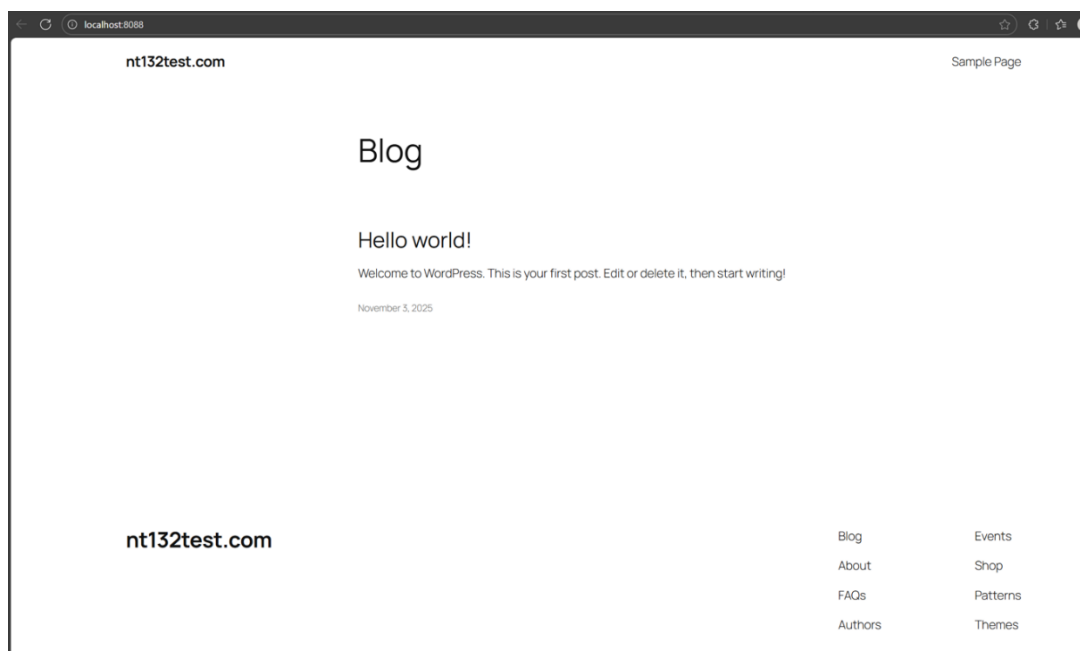
Hình 3.1.7. Đăng kí tài khoản (2)



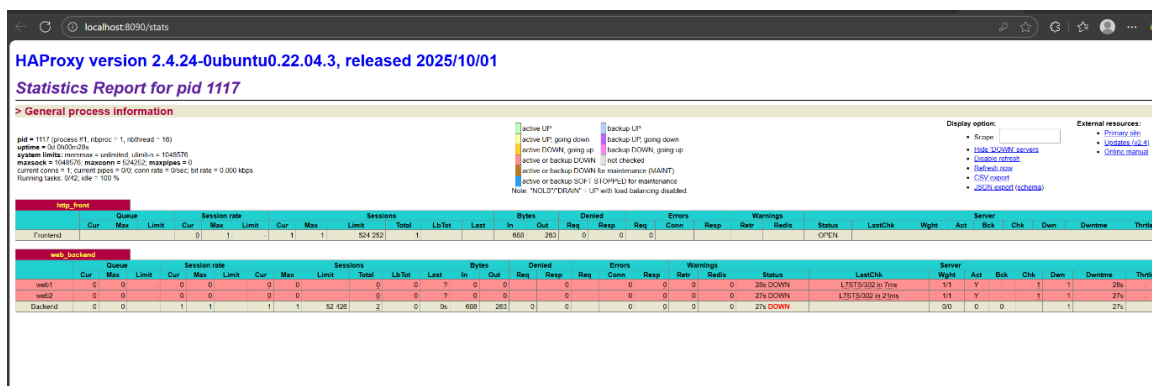
Hình 3.1.8. Dashboard của WordPress



Hình 3.1.9. Đăng nhập



Hình 3.1.10. Giao diện trang web



Hình 3.1.11. Trạng thái của HAProxy

- Với Ubuntu trên VMware:

```
web1@web1-virtual-machine: /mnt/hgfs/ansible
web1@web1-virtual-machine:/mnt/hgfs/ansible$ # Tạo lại thư mục /mnt/hgfs nếu nó không tồn tại
sudo mkdir -p /mnt/hgfs
# Gắn kết (Mount) thư mục chia sẻ từ máy chủ vào /mnt/hgfs
sudo mount -t fuse.vmhgfs-fuse .host:/ /mnt/hgfs -o allow_other,defaults
cd /mnt/hgfs/ansible
sudo ./scripts/vmware-prepare-vm.sh
[sudo] password for web1:
mkdir: cannot stat '/mnt/hgfs': Input/output error
=====
Preparing Ubuntu VM for Ansible
=====
[1/5] Updating system packages...
Hit:1 http://vn.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://vn.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://vn.archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Fetched 129 kB in 2s (78,5 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
278 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
alsa-ucm-conf amd64-microcode apparmor apport apt apt-utils bind9-dnswutils bind9-host bind9-libs bluez
bluez-cups bluez-obexd bubblewrap ca-certificates cups cups-browsed cups-bsd cups-client cups-common cups-core-drivers
cups-daemon cups-filters cups-filters-core-drivers cups-ipp-utils cups-ppd cups-server-common dconf-cli
dconf-gsettings-backend dconf-service dirnmgr distro-info-data dmidecode dmsetup dns-root-data dpkg
fonts-noto-color-emoji fonts-opensymbol ghostscript ghostscript-x gir1.2-gdkpixbuf-2.0 gir1.2-gst-plugins-base-1.0
gir1.2-gstreamer-1.0 gir1.2-harfbbuzz-0.0 gir1.2-javascriptcoregtk-4.0 gir1.2-mutter-10 gir1.2-nm-1.0
gir1.2-packagekitglib-1.0 gir1.2-soup-2.4 gir1.2-udisks-2.0 gir1.2-webkit2-4.0 gjs gnome-control-center
gnome-control-center-data gnome-control-center-faces gnome-shell gnome-shell-common gnome-shell-extension-ubuntu-dock
gnupg gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client gpg-wks-server gpgconf gpgsm gpgv gstreamer1.0-alsa
gstreamer1.0-gl gstreamer1.0-gtk3 gstreamer1.0-packagekit gstreamer1.0-plugins-base gstreamer1.0-plugins-base-apps
gstreamer1.0-plugins-good gstreamer1.0-pulseaudio gstreamer1.0-tools gstreamer1.0-x initramfs-tools
initramfs-tools-bin initramfs-tools-core intel-microcode ipp-usb iputils ping iputils-tracepath libabsl20210324
libapparmor1 libapt-pkg6.0 libarchive13 libblockdev-cryptos2 libblockdev-fs2 libblockdev-loop2 libblockdev-part-err2
libblockdev-part2 libblockdev-swap2 libblockdev-utils2 libblockdev22 libbluetooth3 libc-bin libcapi2-bin
libcapi2-bin libcryptsetup2 libcups2 libcupsfilters1 libcupsimage2 libcurl3-gnutls libcurl4 libdconf1 libdebuginfod-common
```

Hình 3.1.12. Tải và cài đặt gói cần thiết

```
web1@web1-virtual-machine: /mnt/hgfs/ansible
valid_lft forever preferred_lft forever
web1@web1-virtual-machine: /mnt/hgfs/ansible$ ./scripts/vmware-setup.sh
=====
VMware Ubuntu VM Setup Script
=====

This script helps you prepare VMware Ubuntu VMs for Ansible automation.

[1/5] Checking prerequisites...
⚠ Ansible not found. Will install in next step.
[2/5] Setting up Ansible...
[3/5] Installing Ansible collections...
[WARNING]: Ansible is being run in a world writable directory (/mnt/hgfs/ansible), ignoring it as an ansible.cfg source.
For more information see https://docs.ansible.com/ansible/devel/reference\_appendices/config.html#cfg-in-world-writable-dir
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using '--force'.
[4/5] Checking SSH keys...
✔ SSH key already exists
[5/5] Setup instructions...
```

Hình 3.1.13. Setup controller

```
Next steps:
1. Set password for ansible user (optional):
   sudo passwd ansible

2. Copy SSH public key from Ansible controller:
   ssh-copy-id -i <controller_public_key> ansible@<this_vm_ip>

3. Or manually add public key to:
   /home/ansible/.ssh/authorized_keys
   chmod 600 /home/ansible/.ssh/authorized_keys
   chown ansible:ansible /home/ansible/.ssh/authorized_keys
```

Hình 3.1.14. Các bước cấu hình ssh key



```
web1@web1-virtual-machine: /mnt/hgfs/ansible
web1@web1-virtual-machine: /mnt/hgfs/ansible$ sudo passwd ansible
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
web1@web1-virtual-machine: /mnt/hgfs/ansible$ ssh-copy-id -i ~/.ssh/id_rsa.pub ansible@192.168.5.132
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/web1/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
ansible@192.168.5.132's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'ansible@192.168.5.132'"
and check to make sure that only the key(s) you wanted were added.
```

Hình 3.1.15. Cấu hình mật khẩu cho ansible và copy ssh-id cho kết nối

```
web1@web1-virtual-machine: /mnt/hgfs/ansible
web1@web1-virtual-machine:~/Desktop$ cd /mnt/hgfs/ansible
web1@web1-virtual-machine: /mnt/hgfs/ansible$ source ~/.venvs/ansible/bin/activate
(ansible) web1@web1-virtual-machine: /mnt/hgfs/ansible$ ./scripts/vmware-run.sh ping
Testing connection to all VMs...
[WARNING]: Ansible is being run in a world writable directory (/mnt/hgfs/ansible), ignoring it as an ansible.cfg source.
For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
db1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
redis1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
web1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
haproxy1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
web2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
backup1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
(ansible) web1@web1-virtual-machine: /mnt/hgfs/ansible$
```

Hình 3.1.16. Kiểm tra kết nối



```
web1@web1-virtual-machine: /mnt/hgfs/ansible
TASK [backup : Install backup tools] *****
ok: [backup1]

TASK [backup : Ensure cron service running] *****
ok: [backup1]

TASK [backup : Create backup script] *****
changed: [backup1]

TASK [backup : Configure cron job] *****
changed: [backup1]

TASK [backup : Verify cron job exists] *****
ok: [backup1]

TASK [backup : Display cron validation] *****
ok: [backup1] => {
  "msg": "Cron job validation: 1 match(es) found"
}

TASK [backup : Test backup script manually] *****
skipping: [backup1]

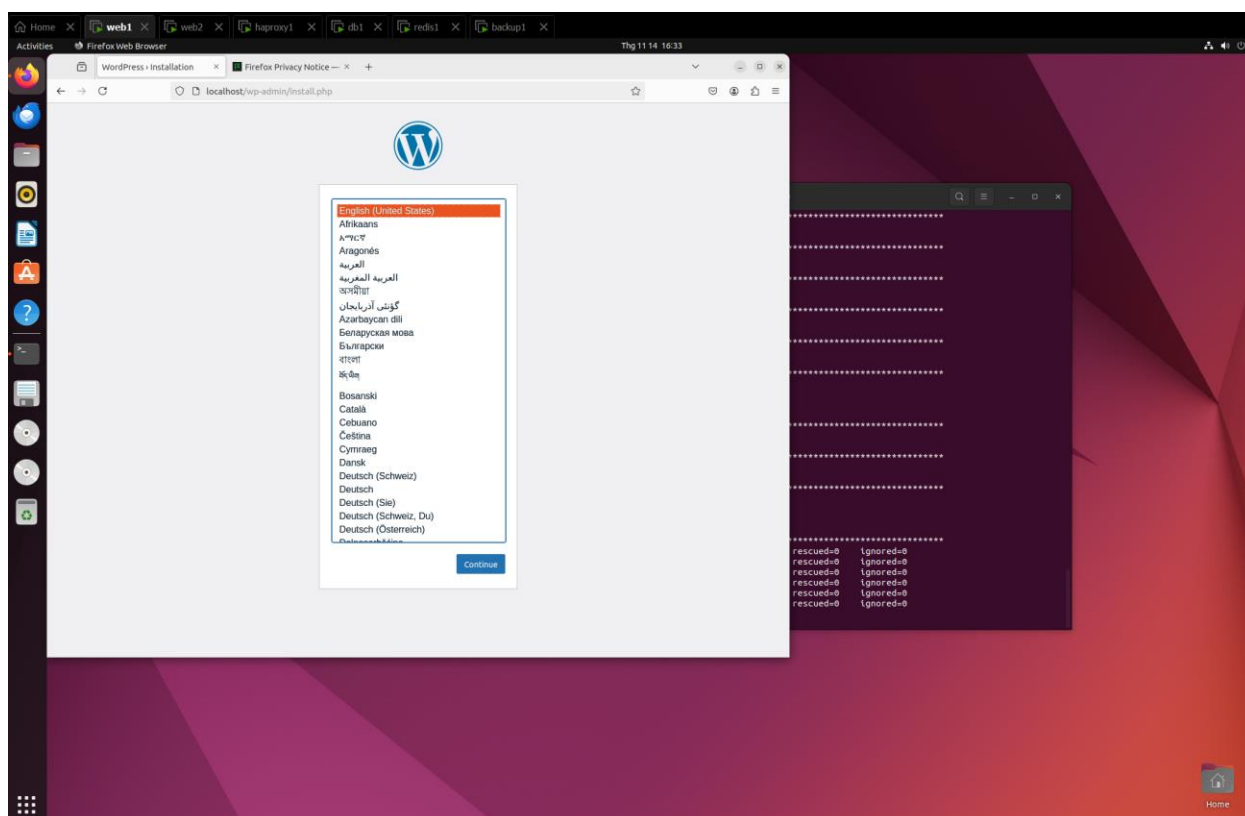
TASK [backup : Check backup directory contents] *****
ok: [backup1]

TASK [backup : Display backup files found] *****
ok: [backup1] => {
  "msg": "Found 0 backup file(s) in /backup"
}

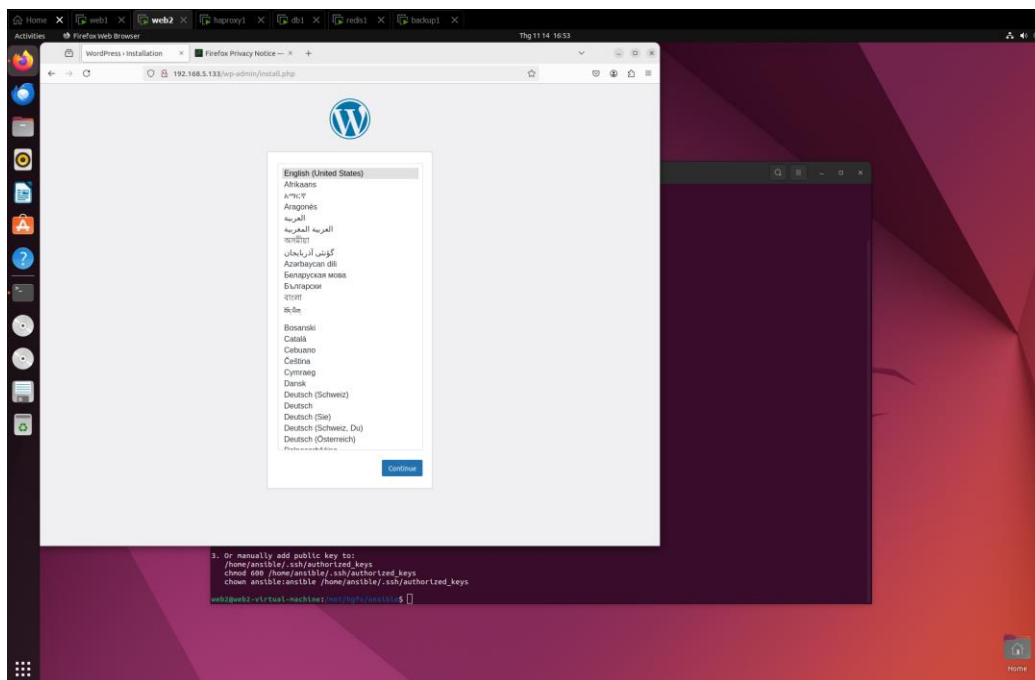
PLAY RECAP *****
backup1      : ok=13  changed=4  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
db1          : ok=12  changed=8  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
haproxy1     : ok=11  changed=5  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
redis1       : ok=8   changed=4  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
web1         : ok=20  changed=14 unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
web2         : ok=20  changed=14 unreachable=0  failed=0  skipped=1  rescued=0  ignored=0

(ansible) web1@web1-virtual-machine: /mnt/hgfs/ansible
```

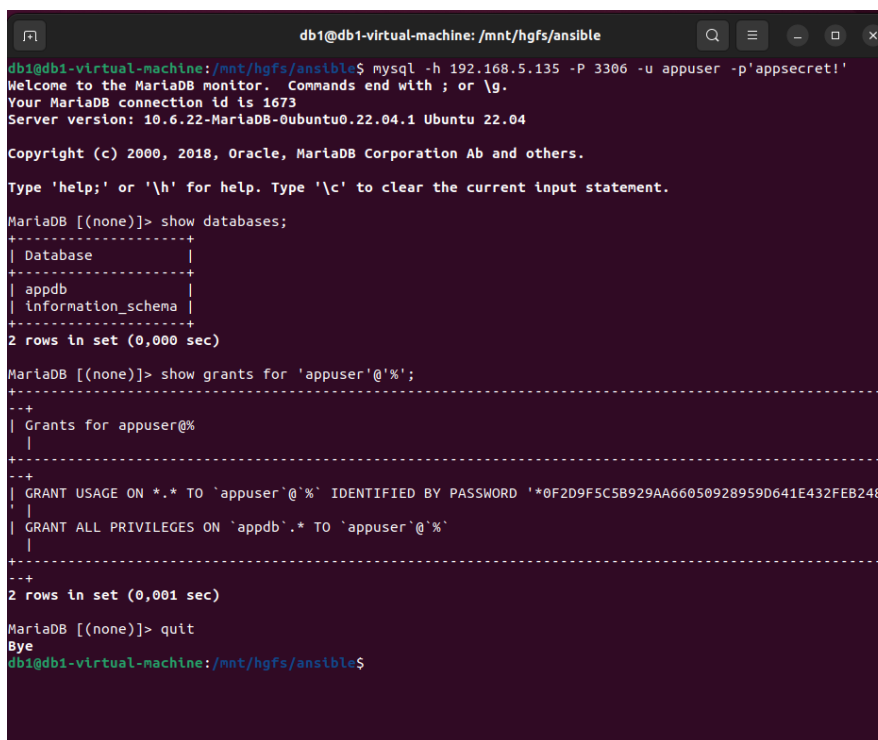
Hình 3.1.17. Kết quả sau khi chạy



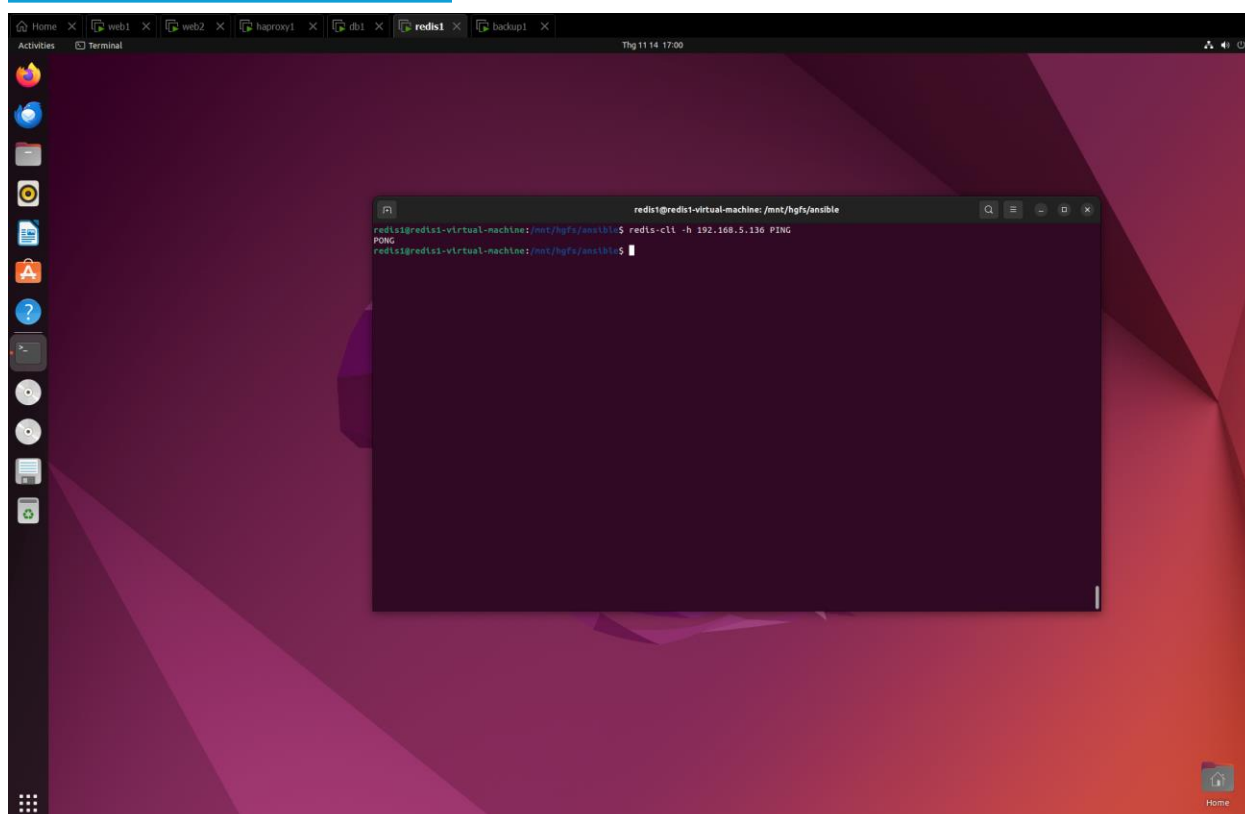
Hình 3.1.18. Localhost test browser



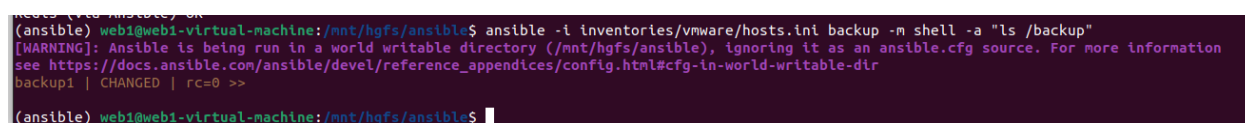
Hình 3.1.19. Kiểm tra bằng ip



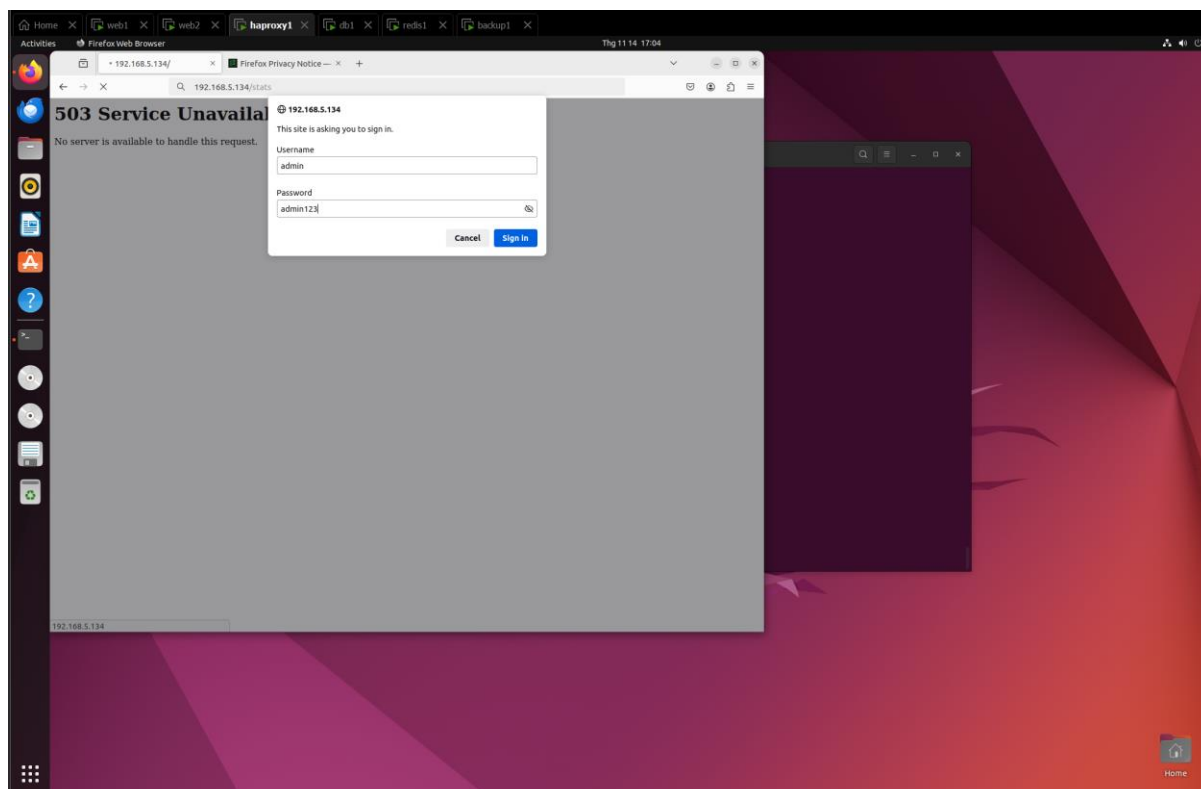
Hình 3.1.20. Kiểm tra cơ sở dữ liệu



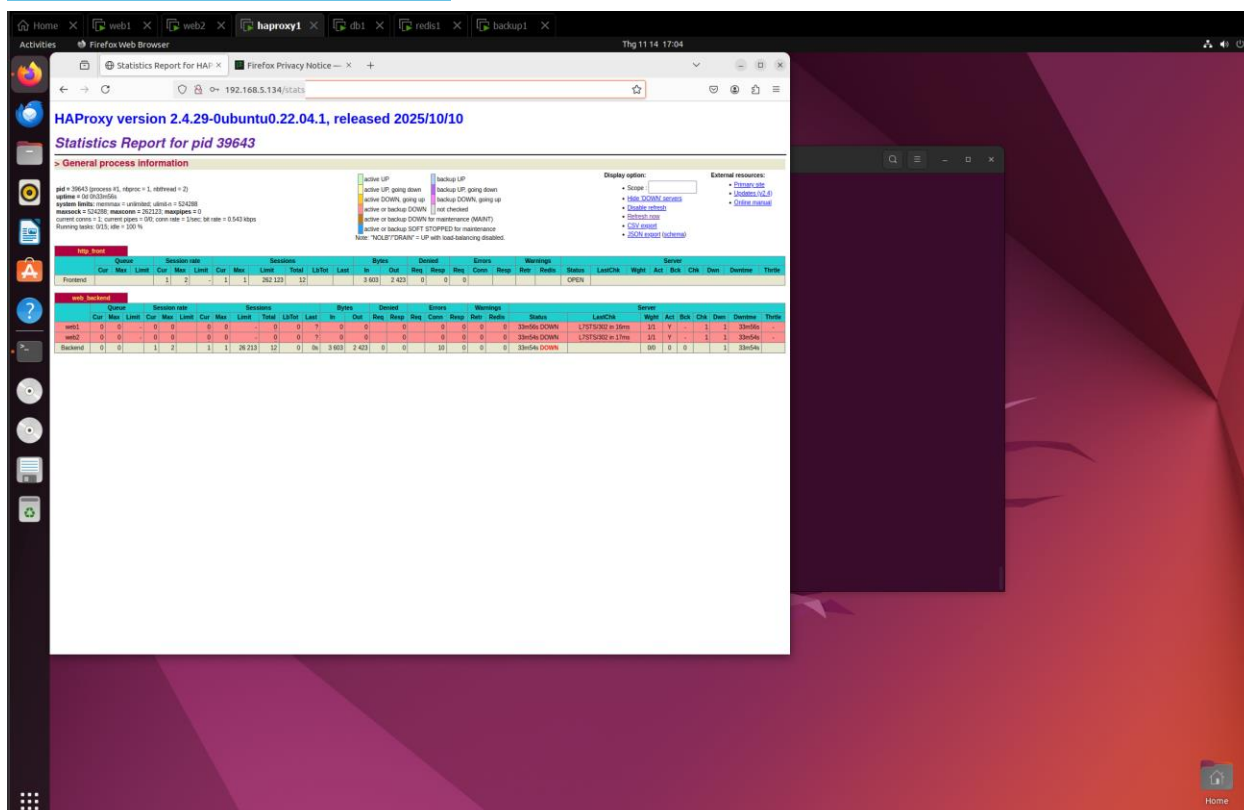
Hình 3.1.21. Kiểm tra redis hoạt động



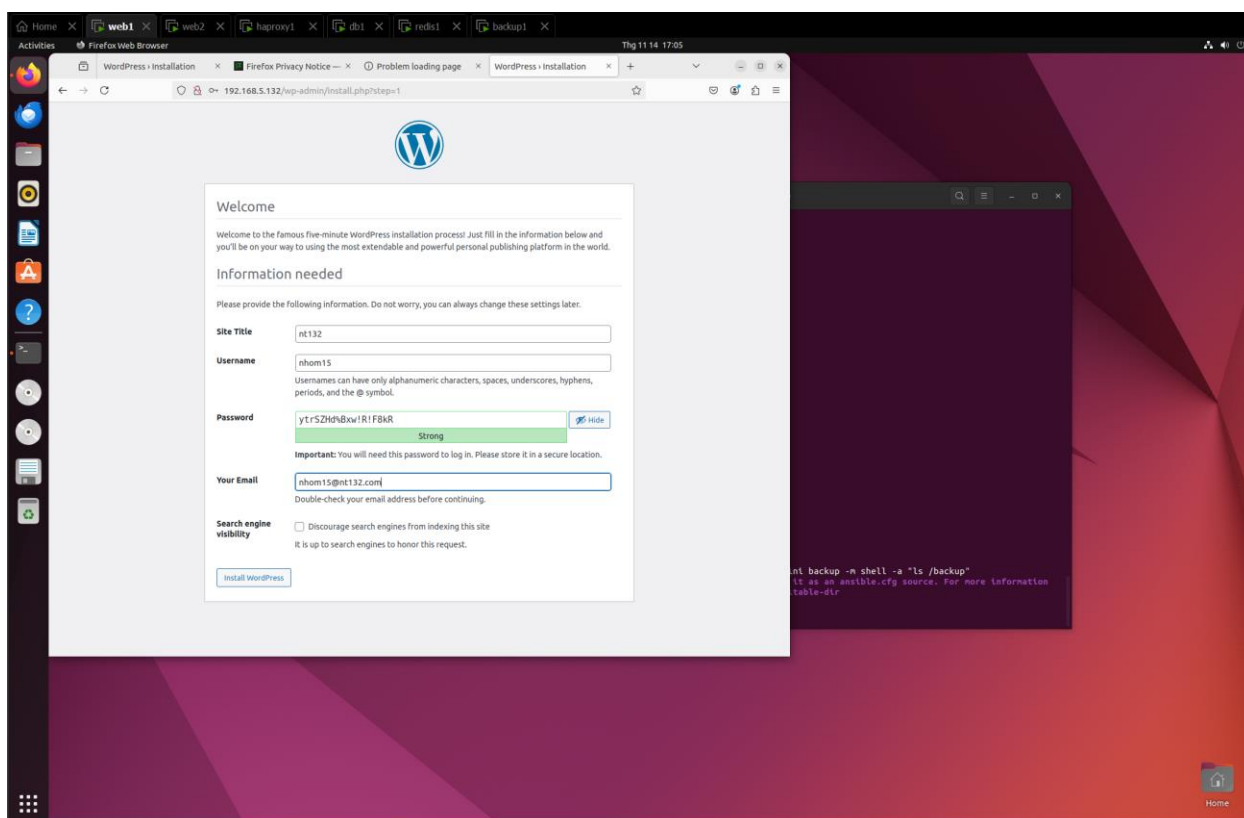
Hình 3.1.22. Kiểm tra backup



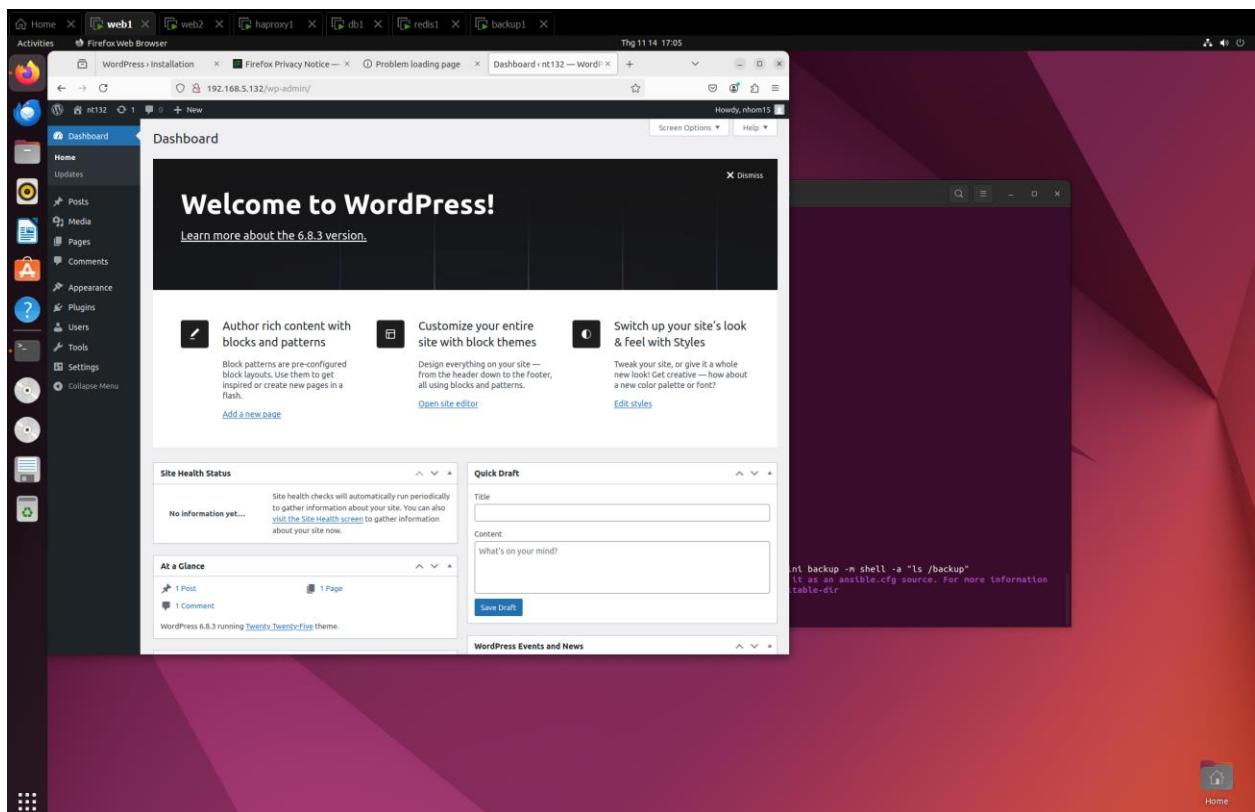
Hình 3.1.23. Đăng nhập user vào haproxy



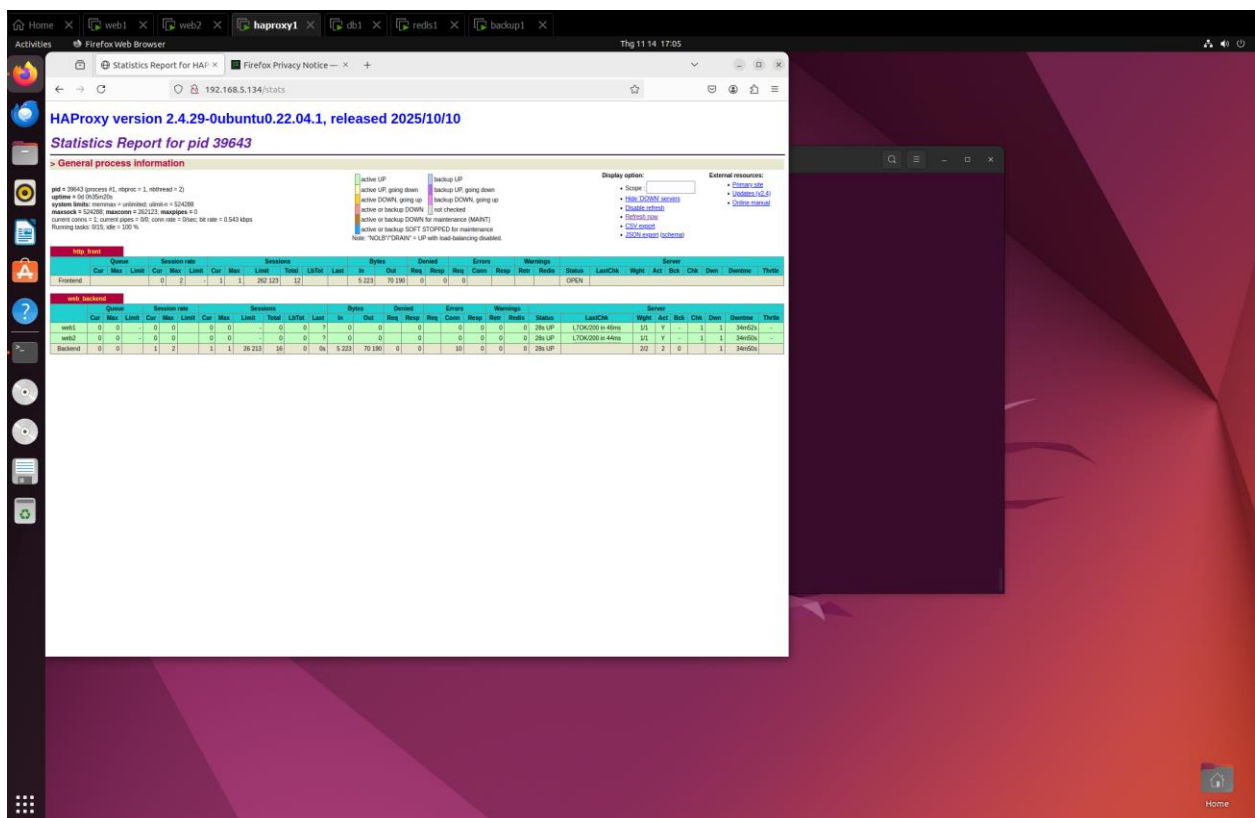
Hình 3.1.24. Trạng thái hoạt động của 2 web server



Hình 3.1.25. Đăng kí



Hình 3.1.26. Đăng nhập



Hình 3.1.27. Trạng thái đã hoạt động trên 2 web server

- Link slide và video demo: [Link slide và video demo](#)

3.2. Kết luận

- Trong khuôn khổ đề tài “Research on Ansible, Install the web service, database and deploy the source code automatically” (Nghiên cứu về Ansible, cài đặt dịch vụ web, cơ sở dữ liệu và triển khai mã nguồn tự động), nhóm đã nghiên cứu và triển khai thành công hệ thống WordPress theo mô hình tự động hóa hoàn chỉnh bằng Ansible. Chi tiết như sau:

- Thứ nhất, nhóm đã xây dựng giải pháp tự động hóa hạ tầng và dịch vụ với Ansible. Toàn bộ quá trình triển khai hệ thống được thực hiện thông qua Playbooks và Roles, bao gồm:
 - Tự động cài đặt và cấu hình 6 máy ảo.
 - Tự động cài đặt và cấu hình các dịch vụ: Web Server, HAProxy, MariaDB, Redis và Backup Server.
 - Tự động triển khai mã nguồn WordPress.
 - Tự động thực hiện backup định kỳ hằng ngày cho dữ liệu quan trọng.
- Thứ hai, hệ thống đạt được tính sẵn sàng cao:
 - Hai Web Servers hoạt động song song, đảm bảo hệ thống vẫn phục vụ khi một server gặp sự cố.
 - HAProxy thực hiện Load Balancing kết hợp Health Check, tự động loại bỏ các Web Server không phản hồi.
 - Database được triển khai độc lập, tránh tình trạng Single Point of Failure ở tầng web.
- Thứ ba, nhóm đã áp dụng thành công mô hình Infrastructure as Code (IaC). Toàn bộ hạ tầng và cấu hình hệ thống được định nghĩa dưới dạng mã nguồn:
 - Các Playbooks và Roles được quản lý bằng Git.
 - Dễ dàng tái tạo môi trường trên hạ tầng mới chỉ với một lệnh chạy Playbook.
 - Giảm phụ thuộc vào thao tác thủ công và kinh nghiệm cá nhân của người quản trị.
- Cuối cùng, hệ thống được đảm bảo tính bảo trì và mở rộng:
 - Code được tổ chức rõ ràng theo từng Role, dễ đọc và dễ kiểm tra lỗi.
 - Có thể dễ dàng mở rộng hệ thống bằng cách thêm Web Server hoặc Role mới.
 - Việc debug và kiểm tra được thực hiện độc lập trên từng thành phần.

4. PHỤ LỤC

4.1. Tự đánh giá

Tiêu chí	Điểm	Tổng điểm (8.75/9)
Báo cáo	4	1
Thuyết trình	3	0.75
Lý thuyết	4	2
Demo	4	5

Bảng 4.1.1. Tự đánh giá

4.2. Phân công

Thành viên	Nhiệm vụ	Mức độ hoàn thành
Vương Thành Đạt	Nghiên cứu về Ansible và soạn demo	100%
Trần Khánh Toàn	Nghiên cứu về Ansible và soạn report	100%
Nguyễn Bá Quân	Nghiên cứu về Ansible và soạn slide	100%

Bảng 4.2.1. Phân công

4.3. Trả lời câu hỏi

Câu hỏi 1	Vậy thì lợi ích thực tế của việc sử dụng ansible là gì so với việc chạy script thủ công? Mình thấy nhóm bạn cũng chia nhiều task khác nhau, nhưng chưa thấy nói gì về việc nếu như có 1 task thất bại thì ansible sẽ kiểm soát nó như thế nào?
Câu trả lời	- So với việc chạy script thủ công, Ansible mang lại nhiều lợi ích rõ rệt: - o Thứ nhất, người dùng chỉ cần mô tả trạng thái mong muốn của hệ thống, Ansible sẽ tự động quyết định có cần thực hiện hành động hay không. Điều này giúp tránh các lỗi phát sinh khi script được chạy nhiều lần. - o Thứ hai, Ansible hỗ trợ cùng một Playbook có thể được chạy nhiều lần mà không làm thay đổi hệ thống nếu trạng thái đã đúng. Ngược lại, script thủ công thường phải tự xử lý và kiểm tra logic, dễ gây sai sót. - Về kiểm soát lỗi, khi một task bị thất bại, Ansible sẽ: - o Mặc định dừng thực thi các task tiếp theo trên host đó. - o Các host khác vẫn tiếp tục chạy nếu không bị lỗi. - o Cho phép cấu hình hành vi bằng các cơ chế như: ▪ ignore_errors ▪ failed_when ▪ block / rescue / always

Câu hỏi 2	Ansible hoạt động theo mô hình agentless như thế nào?
Câu trả lời	- Ansible hoạt động theo mô hình agentless, nghĩa là không cần cài đặt bất kỳ agent hay service nào trên các Managed Nodes. Cụ thể: - o Control Node kết nối đến Managed Node thông qua các giao thức có sẵn: ▪ SSH đối với Linux/Unix ▪ WinRM đối với Windows - o Khi cần thực thi một task, Ansible sẽ: ▪ Truyền module qua kết nối SSH. ▪ Thực thi module trực tiếp trên Managed Node. ▪ Thu thập kết quả và xóa file tạm sau khi hoàn tất.
Câu hỏi 3	Trong file site.yml, các Play chạy tuần tự từ trên xuống dưới. Vậy giả sử Play: Database servers bị lỗi (fail) giữa chừng, thì Ansible có tự động dừng lại không hay nó vẫn tiếp tục chạy tiếp Play: Web servers?
Câu trả lời	- Mặc định, Ansible vẫn tiếp tục chạy Play tiếp. Tuy nhiên, các host trong Play Database bị lỗi sẽ bị đánh dấu là failed và không chạy các task còn lại của Play đó. Ansible cung cấp các cơ chế để kiểm soát hành vi này: - o <i>any_errors_fatal: true</i>: dừng toàn bộ Playbook nếu bất kỳ host nào lỗi. - o <i>max_fail_percentage</i>: dừng khi tỷ lệ host lỗi vượt ngưỡng. - o <i>serial</i>: triển khai theo từng batch để giảm rủi ro. - Như vậy, Ansible không tự động dừng toàn bộ trừ khi người quản trị cấu hình rõ ràng.
Câu hỏi 4	Ở Slide 20, ở file cấu hình, có 2 dòng được gán là false, nhóm có nói là: Không kiểm tra key khi ssh đến để tăng tính linh động hơn? Điều đó có làm cho hệ thống bị mất an toàn hay không?
Câu trả lời	- Việc đặt: <i>host_key_checking = false</i> giúp Ansible không kiểm tra fingerprint của SSH host, từ đó: - o Tránh bị gián đoạn khi triển khai nhanh. - o Thuận tiện trong môi trường cloud hoặc lab nơi máy chủ được tạo/xóa liên tục. - Tuy nhiên, có làm giảm mức độ an toàn, vì: - o Không phát hiện được tấn công Man-in-the-Middle (MITM). - o Không xác thực được danh tính thực sự của host. - Vì thế, chấp nhận được trong môi trường demo, test, lab, CI/CD tạm thời. Còn trong môi trường production thì không nên làm vậy, mà cần xác thực chặt chẽ danh tính máy chủ.

Câu hỏi 5	Khi tự động hóa nhiều server, làm sao tránh race condition hoặc lock file conflict?
Câu trả lời	- Ansible hạn chế race condition bằng nhiều cơ chế: - o Xử lý theo từng host độc lập, tránh ghi chồng chéo tài nguyên. - o Sử dụng serial để triển khai theo từng nhóm nhỏ. - o Modules được thiết kế idempotent, tránh thao tác dư thừa. - o Sử dụng <i>lockfile</i>, <i>run_once</i>, hoặc <i>delegate_to</i> khi thao tác trên tài nguyên dùng chung.
Câu hỏi 6	Làm sao Ansible biết được danh sách địa chỉ IP của các máy cần cài đặt? Nhóm bạn khai báo thông tin này ở đâu?
Câu trả lời	- Ansible lấy danh sách IP và thông tin host từ Inventory, có thể là: - o Inventory tĩnh (file .ini, .yaml). - o Inventory động (script hoặc plugin lấy dữ liệu từ cloud, CMDB, API). - Nhóm mình khai báo thông tin này trong file inventory tên “hosts.ini”.
Câu hỏi 7	Giả sử mình đang đăng nhập vào trang web và request đó được xử lý bởi Web 1. Sau đó mình bấm F5, HAProxy điều hướng mình sang Web 2. Lúc này mình có bị bắt đăng nhập lại không? Tại sao Web 2 lại biết mình đã đăng nhập rồi trong khi này giờ mình làm việc với Web 1?
Câu trả lời	- Bạn sẽ không bị bắt đăng nhập lại vì trong kiến trúc có HAProxy. Khi người dùng đăng nhập tại Web 1 thì session đăng nhập được lưu: - o Trong shared storage (Redis, Memcached, DB), hoặc Trong cookie đã ký (signed cookie / JWT) - o Khi F5 và request được chuyển sang Web 2: ▪ Web 2 đọc session từ shared storage hoặc kiểm tra token. ▪ Nếu session hợp lệ → không cần đăng nhập lại. ▪ Điều này gọi là stateless hoặc shared-session architecture, cho phép hệ thống mở rộng ngang mà không làm gián đoạn trải nghiệm người dùng.

Bảng 4.3.1. Trả lời câu hỏi



TÀI LIỆU THAM KHẢO

- [1] [Cách sử dụng ansible](#)
- [2] [Getting started with Ansible — Ansible Community Documentation](#)
- [3] [\(Phần 1\) Tìm hiểu về Ansible.](#)
- [4] [\(Phần 2\) Tìm hiểu về ansible.](#)
- [5] [Bắt đầu với Ansible: Cài đặt và thiết lập cơ bản - Công ty cổ phần giải pháp Jupitek](#)
- [6] [Ansible Setup Step-by-Step | Beginner's Guide | Automate Configuration Management](#)
- [7] [Hướng dẫn Ansible cho người mới bắt đầu: Playbook, Lệnh & Ví dụ](#)
- [8] [Ansible Là Gì? Hướng Dẫn Cài Đặt Ansible - VinaHost](#)
- [9] <https://omzcloud.vn/resources/blog/ansible/2023-q3-get-started-with-ansible/>
- [10] <https://www.techtransit.org/install-ansible-navigator-execution-environment/>
- [11] <https://www.edureka.co/blog/what-is-ansible/>
- [12] <https://www.liquidweb.com/blog/what-is-ansible/>

HẾT